

The University of Jordan

King Abdullah II School of Information Technology



Project for Software Engineering course

Spring 2026



Supervised by: Prof. Hamad Alsawalqeh

Student Name	Student-ID	Section
Shatha Abu-Rumman (Team Leader)	0233377	6
Sadeen Nababteh	0232012	6
Hala Abu-Shanab	0235346	6
Lujain Alajarmeh	0236908	6
Batool Al-krimen	0238468	6

Executive Summary

ZAD is a mobile application designed to combat food waste by connecting surplus food providers — including restaurants, supermarkets, and hotels — with receiving agencies such as charities, shelters, and community kitchens. The platform manages the full safety lifecycle of perishable donations, ensuring food is transported within safe temperature windows and verified at every handoff through QR code scanning and timestamped chain-of-custody records.

Built using Flutter and Dart for the frontend and Firebase for backend services, ZAD's core capabilities include a dynamic surplus listing engine with color-coded urgency tagging, a smart matching algorithm that pairs providers with the nearest available receivers, GPS-assisted volunteer delivery management, real-time QR code verification for every pickup and drop-off, and multi-role impact dashboards covering provider statistics, receiver inventory, and environmental metrics such as CO₂ diverted and water saved. A gamification system with volunteer rankings encourages consistent participation.

Document Revision History

Version	Date	Author(s)	Description of Change
0.1	March 2026	All Team Members	Initial document creation: Project Initiation (Section 1), Project Management Plan (Section 2), and preliminary stakeholder analysis.
0.2	April 2026	All Team Members	Added Software Requirements Specifications (Section 3): stakeholder analysis, functional requirements, use case scenarios, activity diagrams, sequence diagrams, NFR tables, and data requirements.
1.0	May 2026	All Team Members	Final submission: added system design (Section 4), implementation details (Section 5), executive summary, RTM alignment, and overall document review.

Table of Contents

Executive Summary	2
Document Revision History	3
1.0 Project Initiation	10
1.1 Project overview	10
1.2 Problem Definition.....	10
1.2.1 Current Situation and Problem Statement	10
1.2.2 Issues.....	10
1.2.3 Objectives for each issue	10
1.2.4 General System Requirements.....	11
1.2.5 Constraints.....	11
• Deadline Constraints.....	11
• Budget Constraints	11
• Technical and Functional Constraints	12
1.3 Feasibility Study	12
1.3.1 Technical Feasibility.....	12
1.3.2 Operational Feasibility	12
1.3.3 Economic Feasibility	12
1.3.4 Schedule Feasibility	13
1.3.5 Legal Feasibility.....	13
1.4 Proposed Solution and Expected Project Deliverables	14
1.5 Scope	14
1.6 Local and Global Impact of the Proposed Solution	14
1.7 Naming Conventions and Definitions.....	15
2.0 Project Management plan	15
2.1 Project Organization.....	15
2.2 Roles and Responsibilities	15
2.3 Software Process Model	16
2.4 Tools and Techniques	16
2.5 Work Breakdown Structure.....	16
2.7 Project Schedule.....	18
2.9 Monitoring, Reporting and Control Mechanisms	20

3.0 Software Requirements Specifications (SRS)	20
3.1 System Stakeholders and Requirement Sources	20
3.2 User Requirements and Stories	22
3.2.1 User requirements (informal)	22
3.3 Functional Requirements Specification	23
3.4 Analysis Models	24
3.4.1 Use Case Model	24
3.4.2: Use Cases Scenarios	26
3.4.3 Activity Diagram	29
3.4.4 System Sequence Diagrams (SSD)	31
3.4.5 Domain Model Class Diagram	34
3.5 Non-Functional Requirements	34
3.5.1 Performance and Usability Requirements	34
3.5.2 Dependability Requirements (Reliability & Availability)	35
3.5.3 Security Requirements	35
3.5.4 Usability Requirements	36
3.5.5 Operational and Environmental Requirements	36
3.5.6 Maintainability and Supportability Requirements	37
3.6 Data Requirements	38
3.6.1 Business Data Entities	38
3.6.2 Data Storage and Constraints	38
3.7 Requirements Traceability Matrix (RTM)	38
4.0 Design	40
4.1 System Architecture	40
4.2 Class and object design	41
4.3 Dynamic Modeling (Sequence Diagram)	42
4.3.1 Sequence Diagram: User Login and Authentication	42
4.3.2 Sequence Diagram: Food Donation Posting and Smart Matching	42
4.3.3 Sequence Diagram: QR Code Verification at Pickup and Drop-off	43
4.3.4 Sequence Diagram: Volunteer Driver Accepts Delivery and Navigates to Pickup	43
4.4 Component and Deployment Design	44
4.5 Graphical User Interface (GUI) Design	45

4.6	Database Design (Object to ER Mapping & Schema).....	55
5.0	Implementation	56
5.1	Development Environment	56
5.2	System Architecture Implementation	56
5.3	Graphical User Interface implementation.....	57
5.3.1	Registration page.....	57
5.3.2	Home page.....	58
5.3.3	Donation page	59
5.3.4	Request page	59
6.0	Testing.....	60
6.1	Testing Strategy.....	60
6.2	Test Cases.....	62
7.0	User Manual.....	67
8.0	References	67

List of Figures

FIGURE 1. PROJECT ORGANIZATION.....	14
FIGURE 2. GANTT CHART	17
FIGURE 3. PERT CHART	18
FIGURE 4. USE CASE-1.....	28
FIGURE 5. USE CASE-2.....	29
FIGURE 6. USE CASE-3.....	29
FIGURE 7. SSD 1: LOGIN SCENARIO	30
FIGURE 8. SSD 2: FOOD DONATION SCENARIO	31
FIGURE 9. SSD 3: QR VERIFICATION SCENARIO.....	32
FIGURE 10. MAIN CLASSES	33
FIGURE 11. CLASS AND OBJECT DESIGN.....	40
FIGURE 12. SEQUENCE DIAGRAM: USER LOGIN AND AUTHENTICATION	41
FIGURE 13. SEQUENCE DIAGRAM: FOOD DONATION POSTING AND SMART MATCHING	41
FIGURE 14. SEQUENCE DIAGRAM: QR CODE VERIFICATION AT PICKUP AND DROP-OFF	42
FIGURE 15. SEQUENCE DIAGRAM: VOLUNTEER DRIVER ACCEPTS DELIVERY AND NAVIGATES TO PICKUP	42
FIGURE 16. ADD FOOD DONATION SCREEN	44
FIGURE 17. DONATION DETAILS	44
FIGURE 18. FIND DONATION NEAR ME	45
FIGURE 19. GLOBAL SUSTAINABILITY IMPACT	45
FIGURE 20. LEADERBOARD ACHIEVEMENT	46
FIGURE 21. LOGIN REGISTER	46
FIGURE 22. MY DELIVERY TASK	47
FIGURE 23. MY DONATION TRACKER	48
FIGURE 24. NAVIGATE VERIFY DELIVERY	49
FIGURE 25. PROVIDER DASHBOARD	50
FIGURE 26. RECEIVER DASHBOARD	51
FIGURE 27. RESILIENT SUSTAINABILITY	52
FIGURE 28. SAFETY INCIDENT LOG.....	53
FIGURE 29.....	56
FIGURE 30.....	56
FIGURE 31.....	57
FIGURE 32.....	57
FIGURE 33.....	58
FIGURE 34.....	58
FIGURE 35.....	59

List of Tables

TABLE 1. DEVELOPMENT COSTS	11
TABLE 2. HARDWARE AND SOFTWARE COSTS	12
TABLE 3. OPERATING COSTS	12
TABLE 4. ASSIGNING TEAM MEMBERS TO TASKS	17
TABLE 5. RISK ANALYSIS AND PLANS	18
TABLE 6. SYSTEM STAKEHOLDERS.....	20
TABLE 7. PERFORMANCE AND USABILITY REQUIREMENTS	34
TABLE 8. DEPENDABILITY REQUIREMENTS.....	34
TABLE 9. SECURITY REQUIREMENTS.....	35
TABLE 10. USABILITY REQUIREMENTS.....	35
TABLE 11. OPERATIONAL AND ENVIRONMENTAL REQUIREMENTS.....	36
TABLE 12. MAINTAINABILITY AND SUPPORTABILITY REQUIREMENTS	36
TABLE 13. REQUIREMENTS TRACEABILITY MATRIX.....	38
TABLE 14. DATABASE STRUCTURE (USERS COLLECTION).....	54
TABLE 15. DONATIONS COLLECTION	55
TABLE 16	63
TABLE 17	64
TABLE 18	65

1.0 Project Initiation

1.1 Project overview

ZAD is an application designed to limit and reduce food waste. It focuses on connecting places that have leftover edible food (like restaurants, supermarkets, and hotels) community organizations that need it (like charities and soup kitchens). What makes our application different than standard food delivery apps is that it focuses heavily on food safety, making sure the food stays at the right temperature while it is transported by volunteer drivers.

1.2 Problem Definition

1.2.1 Current Situation and Problem Statement

Nowadays, businesses throw away massive amount of perfectly good food. The main problem that there is no fast, safe, and organized way to transport this extra food from the businesses to the charities before it spoils, while local charities struggle to feed their communities.

1.2.2 Issues

1. There is no coordination between the Surplus Providers and community organizations that need it ; while institutions spend millions to promote their products, there is no real connection to know if they can further take advantage of selling the food to the customer; a lot of customers know people in need or have pets that will be more than exited to eat scraps of chicken.
2. Transporting fresh food can be precarious; as food travel with the volunteers, it may not be kept cold or hot enough to ensure it reaches its destination safe and edible.
3. The businesses donating food and the volunteers driving it cannot track the actual positive results of their hard work, because it's human nature to eagerly see the outcome therefore not a lot of people would volunteer to help.
4. The Lack of Prepping Properly: The real concern is that the Surplus Providers still cannot manage making the proper amount of food, so they end up trashing the excessive amount.

1.2.3 Objectives for each issue

1. Create an efficient and user-friendly online surplus inventory management and communication system to enable organizations to quickly and easily upload excess food, connect, and match with local charities before excess food reaches its expiration date.

2. Ensure that all donated food is transported safely by providing a time-stamped tracking system for all food donations as well as an electronic scan verification system to track the safe delivery of donated food.
3. Provide web-based dashboards that allow both organizations with surplus food and volunteers to quantify the impact of their food donations by providing statistics on the actual amount of food saved through donation and the total number of beneficiaries served through food donations.
4. Provide a set of analytical tools to assist organizations with surplus food to analyze historical donation data to gain insight into food donation trends and reduce the amount of overproduction of food in the future.

1.2.4 General System Requirements

- An easy-to-use interface for providers, Charities & Volunteer drivers so that they can quickly & easily use the application
- The system will have secure User Registration & Login with different access rights assigned for each type of User
- Able to accept a donation from a surplus food provider – including ability to enter the amount of food, category/type of food and expiration date of the food.
- The ability to utilize smart matching between provider and receiver of food.
- Map based on driving directions for volunteer drivers to ensure they make efficient deliveries.
- Use QR-code for verification at pick up and drop off.
- The ability to maintain all User data in a secure, Private manner.
- Must be accessible by using mobile devices.

1.2.5 Constraints

- **Deadline Constraints**

Project deadlines are a key component of successful project completion and effective project management; therefore, all tasks in the Zad project must adhere to a specific timeline during the academic semester, which will require that all team members coordinate and work collaboratively. Meeting deadlines will ensure that the project is completed on time (the implementation phase will be delayed if deadlines are not met), improve the quality of the final product, and provide for the delivery of all required features.

- **Budget Constraints**

Limited finances mean that the project relies heavily on university-supplied open-source tools and materials for development/maintenance work. Development and maintenance activities must be performed in accordance with a well-defined spending plan. Moreover, technical choices should support the system's requirements while remaining within the available budget.

- **Technical and Functional Constraints**

Zad project will be developed to serve primarily as a prototype for the front-end of the system and will not be sold as an actual commercial product. Also, there are several advanced features that cannot be implemented in the system due to the technical limitations of the hardware that will be used to develop, test, and demonstrate the functionality of the system. Zad project is primarily intended for use in an academic setting and is designed to demonstrate the concepts of the overall system and not to deliver an actual fully functional system that could be used in a real-world scenario.

1.3 Feasibility Study

1.3.1 Technical Feasibility

Zad is a mobile app developed by Flutter framework using Dart programming language for frontend functionality which is compatible with android devices. For efficient coding and debugging, we used Visual Studio Code to write the code and debug it, and Android Studio for using emulator to see the result in effective way. To develop a backend system, we used firebase. As hardware equipment, a laptop or computer capable of running the tools for the project. And our team is familiar with these technical skills mentioned above.

1.3.2 Operational Feasibility

Zad is developed to minimize food waste by connecting surplus providers with receiving agencies for easy transporting between them. It allows providers to organize surplus food to impact them tax-deductible donations.

1.3.3 Economic Feasibility

- **Development costs:**

Roles	Hourly pay (JD)	Total work hours	Total cost (JD)
Project Manager	15	50	750

UI/UX Designer	10	30	300
Frontend Developer	20	60	1200
Backend Developer	25	120	3000
Total	5250		

Table 1. Development costs

- **Hardware and Software costs:**

Items	Description	Costs (JD)	Total costs (JD)
Hardware	Laptops Number needed: 2	700	1400
Software	Design and development tools	300	300
Total	1700		

Table 2. Hardware and Software costs

- **Operating costs:**

Services	Cost (JD)	Notes
System Maintenance	400	Cost per year
Firebase services	300	Cost per year
Google play registration	20	Pay once only
Total	720	

Table 3. Operating costs

- **Total estimated cost = 7670 JD.**

Note: these costs are estimated based on market in Jordan.

1.3.4 Schedule Feasibility

We are working hard to make the project applicable and implementable during this semester.

We will use our experiences that we previously learned in the Dart language and the Flutter tool. We will develop the idea into a phone application. We hope to have enough time to design the front-end interaction interfaces. The correct and ideal method that we followed is to distribute tasks among team members based on skills, taking into account equality in the division of work and tasks, and knowing all of us exactly what must be done. Every schedule we create will help us stay organized. And because a complete back-end and databases are not a prerequisite for the project currently, which makes things simpler and an easy way to achieve our goal.

1.3.5 Legal Feasibility

When we study the legal aspect, the idea stipulated in the ZAD application does not violate any rights in the state, but rather addresses a problem that we are suffering from these days. Therefore, this does not mean that we should not be careful and warn against the existing procedures in the state. We emphasize that monitoring food safety is a major priority because it is a surplus of food. Ensuring that it has not expired, examining it, and checking its validity is necessary. Without revealing the user's identity, it is extremely important for us and for them, as this contributes to the design of the system and the protection of sensitive data from unauthorized access. We also include user agreements and the importance of each party in the application to carefully define responsibilities. Overall, our project closely complies with legal and ethical guidelines.

1.4 Proposed Solution and Expected Project Deliverables

It proposes to connect sites (hotels, supermarkets, restaurants) with surplus products and institutions that deliver them to people and families in priority need. We try as much as possible to avoid ambiguity and include details about food in our mobile app, so the system maps it with surrounding areas. And then your delivery company driver can assist and deliver the food, and we handle the order no time lost as a result.

Our project deliverables will include:

1. Flutter functional model.
2. Mobile App Screens (High resolution).
3. Technical diagrams and system documentation.
4. Simple back-end integration (if time allows).

We aim to minimize food waste and contribute to the community.

1.5 Scope

Our system focuses on managing surplus food and ensuring it reaches the right people. This includes:

1. Collecting extra food from the hotels, supermarkets, and restaurants.
2. Matching the food with the recipients (charities, shelters, and community kitchens).
3. Organizing volunteers and drivers on how they take care of the food and what type of vehicle is needed for the food (regular or refrigerated).
4. Providing multi-role impact dashboards for providers (kilos of food saved, tax-deductible donation value), receivers (in-transit inventory), and environmental reporting (CO2 diverted, water saved).

5. A volunteer gamification system featuring badges, rankings, and a “Top Rescuers” leaderboard to encourage consistent participation.
6. An incident reporting and audit log module that records all handoffs and flags shipments rejected for temperature or quality violations.

However, our system does not include:

1. Cooking inside kitchens.
2. Charitable services outside food (like clothing or medicines).

1.6 Local and Global Impact of the Proposed Solution

- **Local impact:**

Reducing food waste in the society, and Supply charities and shelters with safe food resources.

- **Global impact:**

Reducing Greenhouse Gas emissions. Food waste is responsible about 8-10% of greenhouse gas emissions in the world, reducing it directly combats climate change and supports the 2nd goal of sustainable development goals (SDGs) which is (Zero Hunger).

1.7 Naming Conventions and Definitions

- Surplus Food Providers such as hotels, restaurants and supermarkets.
- Receiving Entities such as charities, shelters and community kitchens.
- Volunteers are those who transport the food from the provider to the recipient.

2.0 Project Management plan

2.1 Project Organization

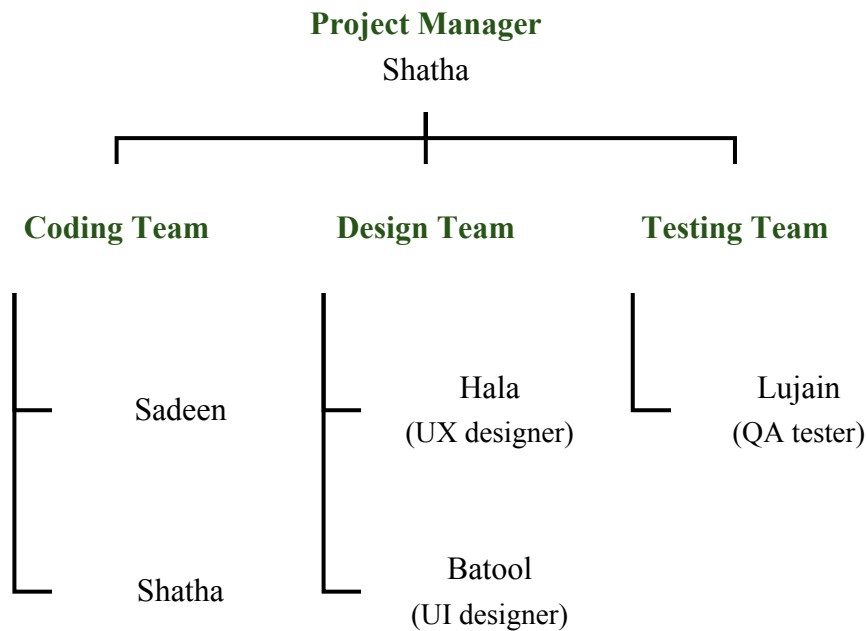


Figure 1. Project Organization

2.2 Roles and Responsibilities

- **Shatha:** is leader. She organized a meeting, distributed the tasks and checked them, she is also a lead programmer. She manages the technical architecture of the project, writes the core code and collaborates with other departments to ensure that the software is built correctly.
- **Sadeen:** is programmer. She is responsible of writing and testing the actual code for the app, she works under the guidance of the lead programmer to fix bugs and make sure that the daily minor technical tasks are completed.
- **Hala:** is UX designer. She focuses on the overall logical flow of the app because she creates the structural blueprint that's why she works user needs and builds wireframes, as she maps out the step by step journey from a restaurant listing the donated food organizations to a charity organizing a pickup all while ensuring the platform is logical, accessible and straight forward to navigate .
- **Batool:** is UI Designer. She works on the visual appearance of the application, she chooses the appropriate colors, lines and designs buttons and tables to ensure “Zad” is eye catchy.
- **Lujain:** is a QA Tester. Her responsibility is to ensure the app has no bugs by running different test cases that provides the users with an app that runs smoothly.

2.3 Software Process Model

The team will work using the Waterfall model for the following reasons. First, the project requirements were gathered upfront through a structured interview with a domain expert and are clearly defined and documented in the SRS before implementation began, which aligns with Waterfall's requirement for stable, well-understood requirements. Second, the project has a fixed academic deadline with defined deliverables per phase (initiation, requirements, design, implementation), making the sequential phase structure of Waterfall a natural fit. Third, since ZAD is developed as a prototype focused on demonstrating a specific, bounded set of features, there is limited scope for iterative discovery that would justify an Agile approach. Finally, the team size is small (five members) with clearly assigned roles, making the linear, document-driven workflow of Waterfall manageable and traceable. While Agile would offer more flexibility to adapt to changing requirements, the fixed scope, clear upfront requirements, and academic timeline make Waterfall the most appropriate choice for this project.

2.4 Tools and Techniques

Our team has decided to use the following tools and techniques for the ZAD project:

- Development Tools: Visual Studio Code, Android Studio and GitHub.
- Design Tools: Lucidchart and Figma.

2.5 Work Breakdown Structure

Project Tasks

- Project Initiation
- Design
- Development
- Testing
- Presentation

Task Definition

- **Project Initiation**
 - Define the problem, solution and gather requirements (3 days)
 - Prepare a project management plan (4 days)
 - Write the feasibility study (1 days)
 - Document software requirements (2 days)
- **Design**
 - Design GUI prototypes using Figma (5 days)

- Design software architecture and UML in Lucidchart (4 days)
- **Development**
 - Implement the Mobile UI and features using Flutter and Dart (3 weeks)
- **Testing**
 - Test all app components and fix bugs (2 weeks)
- **Presentation**
 - Prepare the final report and the presentation

Deliverables and Milestones

- Milestone 1: start and initiate the project.
 - Deliverables: submit the initial project proposal.
- Milestone 2: complete project plan and start UI design.
 - Deliverables: UI mockups and design document.
- Milestone 3: complete development and implementation.
 - Deliverable: The Flutter application source code and final documentation.
 - Deliverable: give a final presentation and submit the complete software requirements specification report.

Resources Needed

- Skills: Time management, teamwork, (Flutter and Dart) development and UI/UX design skills.
- Hardware: Laptops with high performance (minimum 8GB RAM to run Android Emulator and VS Code)
- Software: Google Meet for meetings, Figma for design, VS code (IDE), GitHub and Lucidchart.

Dependencies and Constraints

Dependencies: The team must complete the UI design before starting the coding phase in Flutter.

Constraints: The project must be completely functional and ready for delivery before the final deadline in May.

2.6 Assigning Team Members to Tasks

	Task Title	Task Owner	Duration
1	Project Initiation		
T1	Define the problem and gather	All Team	3 days

	needed data and requirements		
T2	Prepare a project management plan	Shatha	4 days
T3	Write the feasibility study	Sadeen/Shatha	1 day
T4	Document Software Requirements	Lujain	2 days
2	Design		
T5	GUI Prototyping	Batool	5 days
T6	Design Software architecture	Hala	4 days
3	Development		
T7	Implement GUI	Sadeen/Shatha	3 weeks
4	Testing		
T8	Testing all components	Lujain	2 weeks

Table 4. Assigning Team Members to Tasks

2.7 Project Schedule

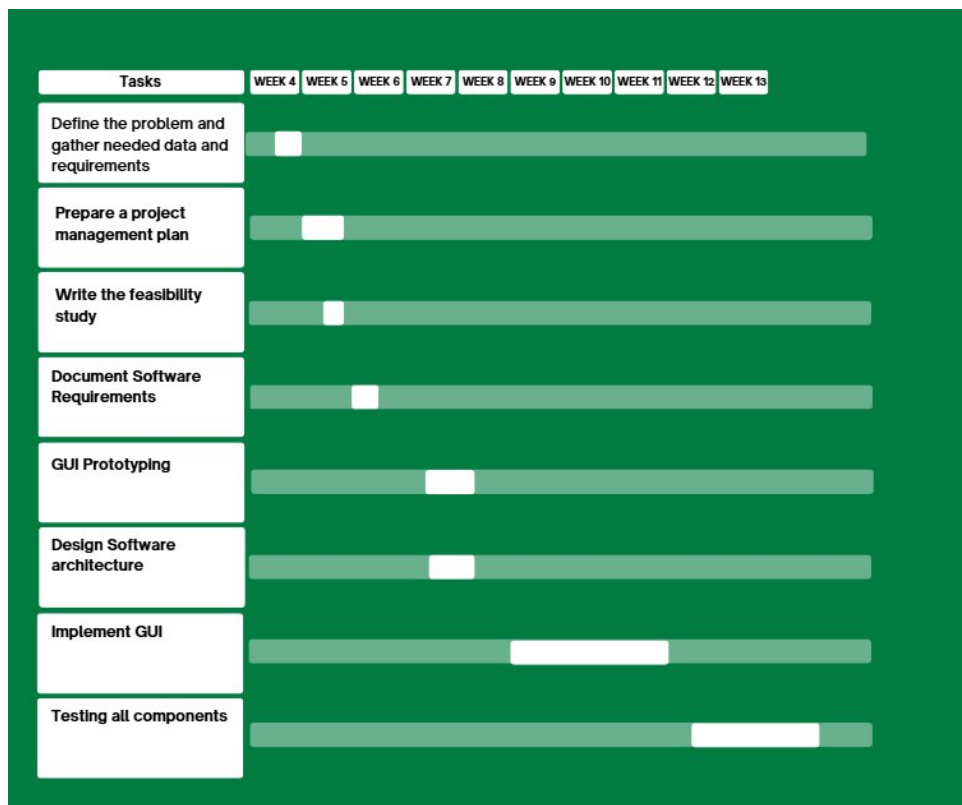


Figure 2. Gantt Chart

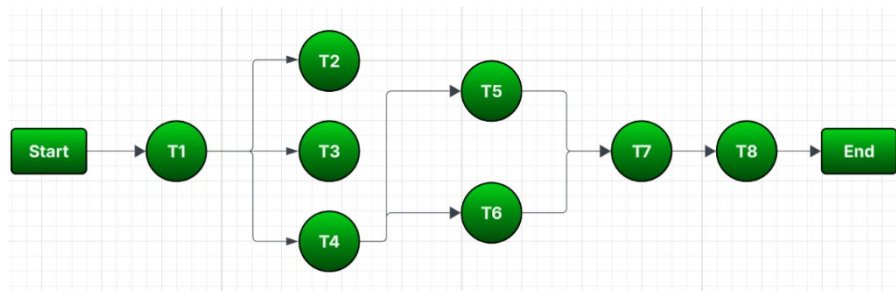


Figure 3. PERT Chart

2.8 Risk Analysis and Plans

Risk Identification	Risk Probability	Risk Evaluation	Risk Mitigation Plan
Tight deadline for project completion	High	Serious	Follow the project schedule carefully and conduct regular team meetings to track progress and ensure tasks are completed on time.
Lack of coordination between team members	Moderate	Serious	Assign clear roles and responsibilities and maintain continuous communication between team members.
Difficulty integrating system features such as QR-code tracking and routing logic	Moderate	Tolerable	Simplify implementation features when needed and focus on prototype-level functionality.
Limited experience with some development tools or technologies	Moderate	Tolerable	Use available learning resources, tutorials, and instructor guidance to improve

			technical skills during development.
Changing or unclear system requirements	Low	Tolerable	Define system requirements clearly at the early stages and review them periodically with the team.
Data loss or documentation errors during development	Low	Serious	Maintain backup copies of project files regularly using cloud storage or version control tools.

Table 5. Risk Analysis and Plans

2.9 Monitoring, Reporting and Control Mechanisms

Project monitoring mechanisms:

- Regular (every 1-2 weeks) team meetings to review completed tasks, discuss current progress and plan future activities.
- Team members report continuously on current tasks to ensure effective coordination and collaboration.
- Control the schedule by completing all project phases within the deadlines stipulated in the project management plan.
- Control quality by reviewing all deliverables to ensure they meet the academic and technical standards specified in the project documentation.
- Control implementation by evaluating all new features added during development and their impact on existing system components' stability.
- Track milestones to ensure project objectives are accomplished in a timely manner and aid in monitoring project progress through regularly defined clear milestones.
- Control documentation by continually updating project documentation with new progress and all changes made during development.

3.0 Software Requirements Specifications (SRS)

This section specifies all functional and non-functional requirements of the “Zad” system. It defines the system users, their needs, and the behavior of the system.

3.1 System Stakeholders and Requirement Sources

ST ID	Stakeholder	explanation
ST1	surplus (User-1)	Primary users who want the food to be delivered to receiving agencies
ST2	Receiving agencies (User-2)	Primary users who will receive the extra food
ST3	Volunteers (optional user)	They will help to transfer the food and pack it
ST4	System Administrator	Manages user accounts, roles, permissions, and ensures system security and stability.
ST5	Development & IT Team	Responsible for building, maintaining, and updating the system.
ST6	UI/UX Designers	Design a simple, clear, and friendly user interface.
ST7	Testing team	Responsible for ensuring the system quality before the release.

Table 6. System Stakeholders

o Requirement Sources:

1. Interviews

A food logistics and retrieval coordinator was interviewed to understand the key challenges related to redistributing leftover food, food safety, and coordinating delivery operations. The aim of the interview was to collect true information from users to decide precise requirements for the Zad system.

Interviewee: Mr. Ahmed Al-Ahmad – Food Retrieval and Logistics Coordinator

Interview: Fadia

Date: 1st/April/2026

Fadia: Thank you for your time. Our company is developing a smart food redistribution application. So, I would like to understand the biggest problems that organizations like (hotels and restaurants) face when dealing with surplus food donations. What are the most common problems?

Mr. Ahmed: One of the biggest problems is the time factor. Foods are short-lived, especially prepared meals and dairy products. Often, by the time we find a grantee or arrange transportation, the food is about to expire or is unsafe for eating.

Fadia: Do restaurants or hotels have difficulty tracking the condition of food during transportation?

Mr. Ahmed: Yes, of course. Food safety is important. Some foods require refrigerated transport, but we often do not know if the temperature is kept within safe limits during delivery. Without proper tracking, there is always a risk of accidentally distributing spoiled food.

Fadia: What difficulties do you face when matching donations with charities or shelters?

Mr. Ahmed: The matching process is often done manually through phone calls or groups of messages, and this wastes a lot of time. Sometimes, a shelter receives more food than it can store it, while another nearby shelter still needs food. Having an intelligent system that considers distance, capacity, and importance will make the process faster and more efficient.

Fadia: Will real-time tracking of deliveries and volunteer drivers help you?

Mr. Ahmed: Of course. Real-time tracking will enable us to know the location of food load and whether it will arrive before its expiration date. It will also help us arrange volunteer work more effectively and reduce delivery delays.

Fadia: The system includes urgent measures that change color as the food approaches its expiration date. Do you think this feature will be useful?

Mr. Ahmed: Very useful. Employees handle a large number of donations daily, so visual urgency measurement will help them decide which foods to deliver first before it expire.

Fadia: In your opinion, what are the most important features that should be available in this type of system?

Mr. Ahmed: I think these characteristics should be a high priority:

- 1.Simple entry of surplus food quantities with expiration date tracking
- 2.Smart matching between donors and beneficiaries
- 3.Instant tracking of connection and temperature
- 4.Coordination of volunteers responsible for delivery
- 5.Urgent notifications and alerts for foods near expiration
- 6.Control panel to track donations and delivery status

If the system includes these features, it will significantly contribute to reducing food waste and improving food distribution efficiency.

Fadia: Thank you a lot. Your feedback has helped us to understand system requirements and functional challenges more clearly.

Mr. Ahmed: You are very welcome. I hope this project succeeds because it can make a real difference in helping the community and reducing food waste.

3.2 User Requirements and Stories

3.2.1 User requirements (informal)

The system should allow users to create accounts and log in securely according to their role (surplus food provider, beneficiary, delivery volunteer, or manager).

Surplus food providers such as restaurants, supermarkets, and hotels should be able to create and manage food donation lists by entering food details such as food type, quantity, preparation time, storage requirements, and expiration time. They should also be able to track the status of their donations and receive notifications when a donation is accepted, received, or delivered.

Receivers such as charities, shelters, and community kitchens should be able to browse and accept available food donations based on their needs and storage capacity. they should also be able to track deliveries instantly and receive alerts for urgent donations near expiration.

Volunteers and delivery drivers should be able to view their assigned delivery tasks, access pickup and delivery locations, and update delivery status during transportation. The system must also support real-time GPS tracking and temperature monitoring of sensitive foods to ensure food safety during delivery.

3.2.2 User Stories

Provider User Stories:

- As a surplus food provider, I want to create an account and log in securely, so I can safely manage my food donations.
- As a surplus food provider, I want to add lists of surplus food with expiration details, so beneficiaries can know the condition of the food and how urgent it is.
- As a surplus food supplier, I want to add the quantity details and storage guidelines, so that the transportation process is handled properly.
- As a surplus food provider, I want to receive notifications when a hand-out is accepted by a beneficiary
- As a surplus food provider, I want to track the delivery status of my donations, so I know when the food reaches the recipient.
- As a surplus food provider, I want to have urgency measures for food that is near expiration, so I can prioritize these donations before they become unusable.

Receiving User Stories:

- As a receiver, I want to browse available food donations, so I can order food that suits our needs.
- As a receiver, I want to receive food suggestions based on our storage capacity and location,
- As a receiver, I want to track food deliveries in real time, so I know when the load will arrive.
- As a receiver, I want to receive alerts for urgent food donations, so we can quickly accept food before it expires.

- As a receiver, I want to view donation details and food safety information, so I can make sure the food is safe to use.

Stories of the volunteer user responsible for delivery:

- As a delivery volunteer, I want to log into the system and view my delivery tasks, so I can manage transportation tasks efficiently.
- As a delivery volunteer, I want to access pickup and delivery locations via maps, so I can deliver food quickly.
- As a delivery volunteer, I want to update the delivery status instantly, so food providers and receivers can track the load.
- As a delivery volunteer, I want to receive notifications of urgent deliveries, so that food reaches beneficiaries before it expires.
- As a volunteer responsible for delivery, I want the system to monitor the temperature of transporting sensitive foods, so that food safety standards are maintained.

3.3 Functional Requirements Specification

1. Authentication Module

- The system shall allow users to create an account using name, email, password and account type.
- The system shall allow registered users to log into the system securely using email and password.
- The system shall allow users to reset forgotten passwords.
- Each user is directed to a dashboard that matches their role.

2. Provider Module

- The system shall allow providers to add surplus food donations with more details, like food type, quantity, preparation time and expiration time.
- The system shall allow providers to upload food images.
- The system shall allow providers to view all donations.
- The system shall display dashboard statistics including Kilos of Food Saved, Value of tax-deductible donations and Environmental dashboard.

3. Receiver Module

- The system shall allow receivers to view available food donations.
- The system shall match receiver with nearest provider based on population count.
- The system shall display “In-Transit” donations to receivers.

4. Volunteer Module

- The system shall allow volunteers to register their vehicle type.

- The system shall allow volunteers to update their availability status.
- The system shall assign delivery tasks to available volunteers.
- The system shall allow volunteers to scan QR codes during pickup and delivery.
- The system shall calculate volunteer rankings based on completed deliveries.

5. QR & Tracking Module

- The system shall generate unique QR codes for each donation.
- The system shall verify pickup and delivery operations using QR code scanning.
- The system shall record timestamps for all food hand-off operations.

3.4 Analysis Models

3.4.1 Use Case Model

- UC-1:

The user case name is listing surplus food.

The description is that the food is listed by the providers taking into consideration critical metadata with strict expiration countdowns, providers should add “Urgency Tags” that change color as the food becomes near its safety limit.

- UC-2:

The user case name is matching and routing.

The description is that the food that’s donated is automatically matched to the nearest hungry “Receiver” based on their current capacity and population count therefore it calculates the most efficient routes for drivers to pick up multiple small donations in one trip to reduce carbon emissions.

- UC-3:

The user case name is logging temperature and QR verification.

The description is that providers log storage temperature before and after hand- off and this should be a requirement, and the drivers must scan QR codes at pick up and at drop off to prove the “food travel time” did not exceed safety regulations.

- UC-4:

The user’s case name is stakeholder impact dashboards.

This description consists of three parts:

First of all, Provider Dashboard it tracks “kilos of food saved” and calculates the financial value of tax-deductible donations.

Then we have receiver dashboards it contains inventory management for kitchens to see what supplies “In-Transit” are so they can plan meals in advance.

Finally, we have environmental dashboard it visualizes the “water saved” and the “CO2 Diverted” by preventing food from reaching landfills.

UC-5:

The user case name is volunteer and fleet management.

The description is that it shows the vehicle type whether its refrigerated or not.

And the available scheduling as part of onboard workflow for volunteer drivers not to forget the gamification system that has badges and rankings for “Top Rescuers” to encourage consistent participation.

UC-6:

The user case name is Incident reporting.

The description maintains a permanent, unalterable log of all hand-offs for municipal health inspections and insures automatic "Incident Reporting" if a shipment is rejected due to temperature or quality issues.

3.4.2: Use Cases Scenarios

- UC-1:

The primary actors in this user case scenario are different surplus providers such as hotels, supermarkets and restaurants

The precondition is that the providers should be authenticated and logged into the platform

The main success scenario:

1. The provider should select “create new listing”.
2. Then the provider inputs the food metadata with includes type, quantity and preparation time.
3. After that the provider sets a strict expiration countdown.
4. Then the system applies an “Urgency Tag” (ex. Green for fresh).
5. After that the listing of the donated food is published to the dynamic surplus listing engine.

The Alternate Flow is that the Urgency Tag immediately turns red to signal high priority that means the expiration time is less than 60 minutes.

The exception that if critical metadata (like preparation time) is missing then the system prevents publishing.

○ UC-2:

The primary actors in this user case scenario is the system and its automated

The preconditions are that the automated system displays that a new surplus listing has been published and that one or more receiving agencies have active capacity to receive the donated food.

The main success scenario:

1. The algorithm is designed to identify the nearest hungry “receiver” based on their current population count.
2. Then the system calculates the fastest route for the available drivers.
3. After that the system puts multiple small food into one trip to reduce carbon emissions.
4. Finally, a notification is sent to the assigned volunteer courier.

The alternate flow is that because there is no volunteer available the system expands its search radius to find the next available courier.

The exception is that if the nearest receiver is at full capacity, then the system matches the donation to the next closest agency.

○ UC-3:

The primary actor in this user case scenario is the surplus providers

The precondition is that a match and route task has been assigned

The main success scenario:

1. First the provider receives the notification that the driver is on his way.
2. Then the provider measures the current storage temperature of the food.
3. After that the provider logs the temperature into the “digital health passport”.
4. Then the system confirms that the temperature is within the “safe temperature window”.
5. Finally, the system generates a unique QR code for the hand-off.

The exception is that if the temperature is not within the safe window, the system cancels the pickup and triggers UC-6.

- UC-4:

The primary actor in this user case scenario is the volunteer courier

The preconditions are that the volunteer has accepted a delivery task through the smart matching algorithm and the provider completed the digital health passport entry.

The main success scenario (these are the main steps):

1. First of all, the volunteer arrives at the provider's location.
2. Then the volunteer inspects how the food is packaged for quality.
3. After that the volunteer scans the providers unique QR code to start the "Chain of Custody".
4. Then the system records the exact timestamp and starts coordinating the GPS.
5. After that the volunteer transports the food within the safe temperature window.
6. Then the volunteer arrives at the agency that receives the food and scans the receivers' QR code.
7. Finally, the system confirms that the travel time of the food did not exceed the safety limits and marks the food delivery as "Saved".

The alternate flow is if the route involves multiple small donations, the volunteer repeats steps 1-4 for each stop before continuing to the receiver.

The exceptions are that if the "Time-Stamp Verification" shows that the travel time exceeds safety regulations, the system triggers an automatic "Incident Report". Another exception that if the receiver identifies quality issues at the drop-off, the shipment is marked as "Rejected" and logged for disposal to a landfill.

- UC-5:

The primary actor in this user case scenario is the stakeholders whether it's the provider, receiver or admin.

The precondition is that every stakeholder is logged into the dashboard.

The main success scenario:

1. First of all, the stakeholder selects the "Environmental Dashboard"
2. Then the system retrieves data on "kilos of food saved" and "water saved"
3. After that system visualizes "The CO2 Diverted" by not letting food reach landfills.
4. Finally, the stakeholder views real-time statistics and financial value of donation.

The alternate flow is when the volunteer views their rankings and “Top Rescuer” badges via the gamification system.

- UC-6:

The primary actor in this user case scenario is automated system.

The precondition is that a shipment can be rejected at any stage whether its pickup, transit, or drop-off.

The main success scenario:

1. The system detects failure whether it's a temperature violation or quality rejection.
2. Then the system generates an automatic “incident Report”.
3. After that the system creates a permanent, unalterable log entry for municipal health inspections.
4. Then the system orders the courier to dispose the items at the landfill.
5. Finally, a notification is sent to all relevant stakeholders regarding the failure.

The exception is if an inspector requests logs, the system exports the incident report history as a verified document.

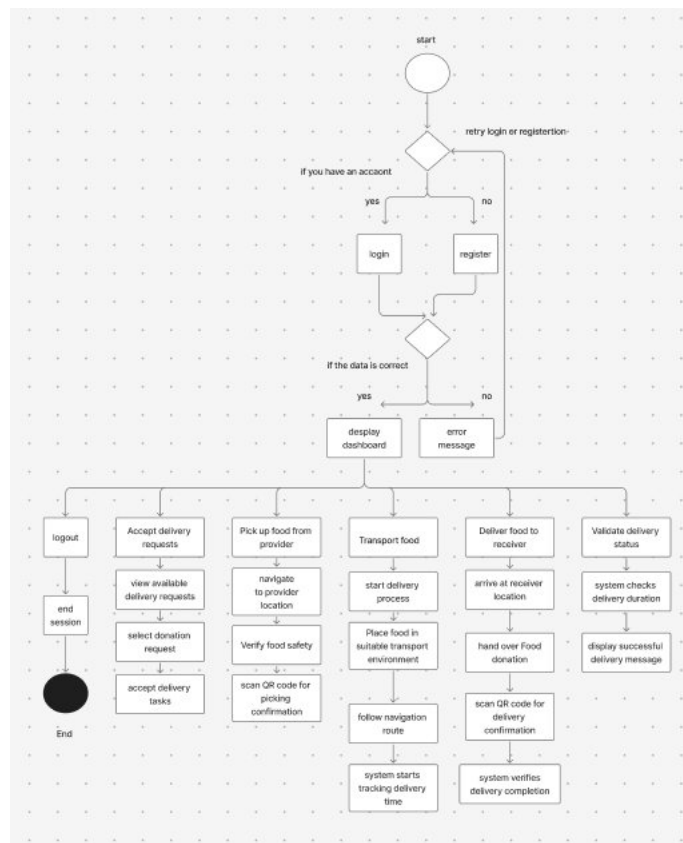


Figure 4. Use Case-1

3.4.3 Activity Diagram

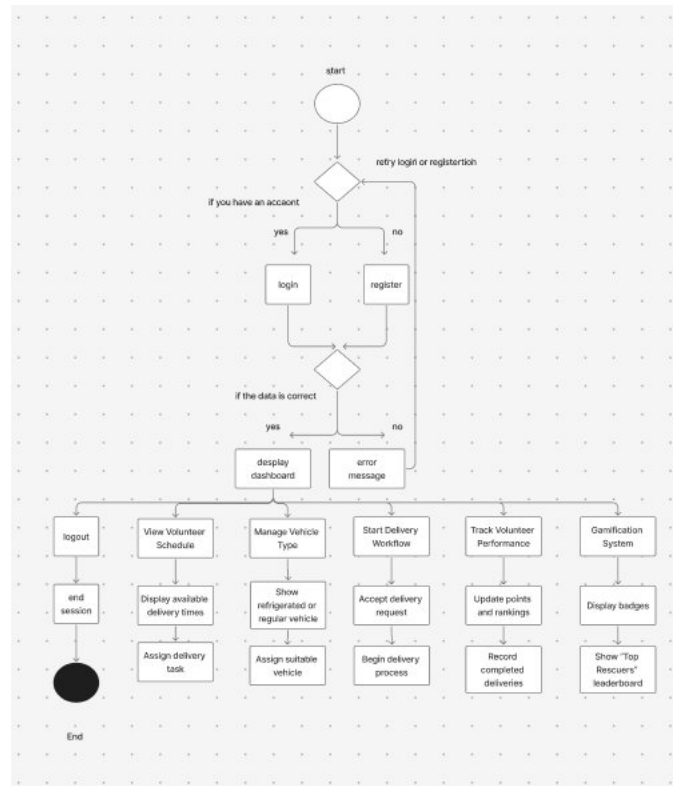


Figure 5. Use Case-2

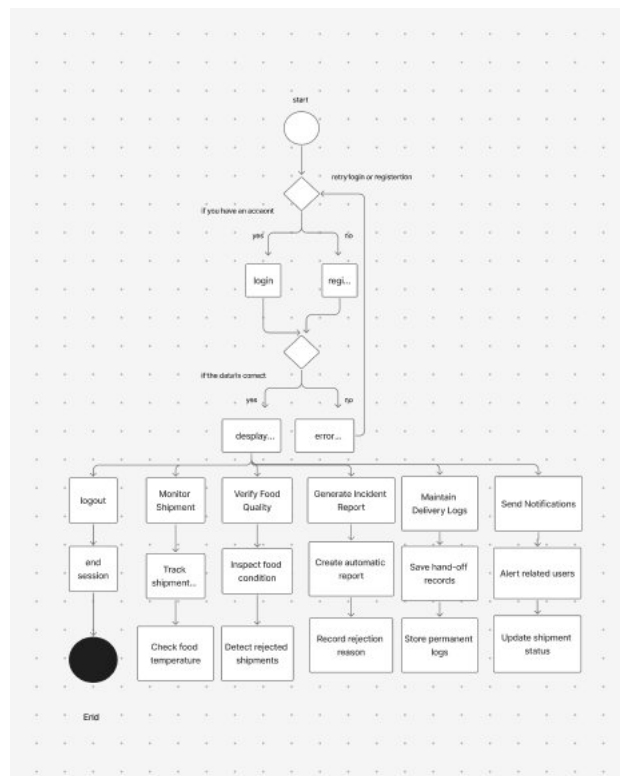


Figure 6. Use Case-3

3.4.4 System Sequence Diagrams (SSD)

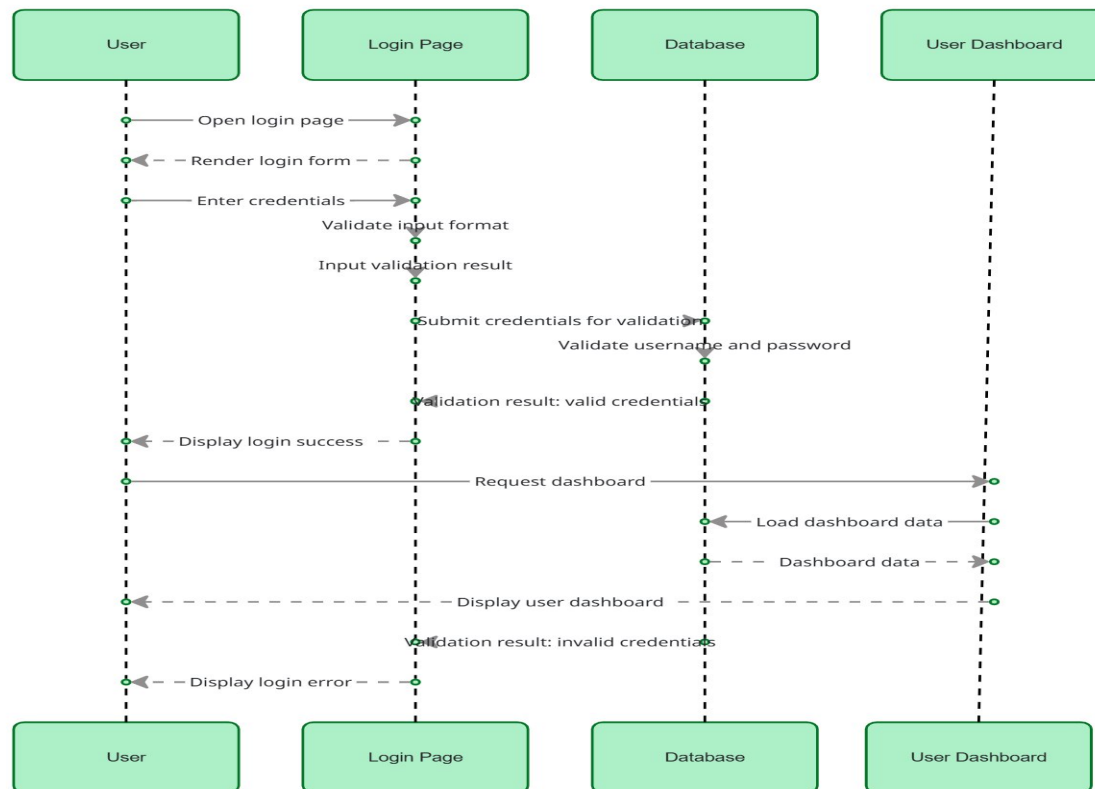


Figure 7. SSD 1: Login Scenario

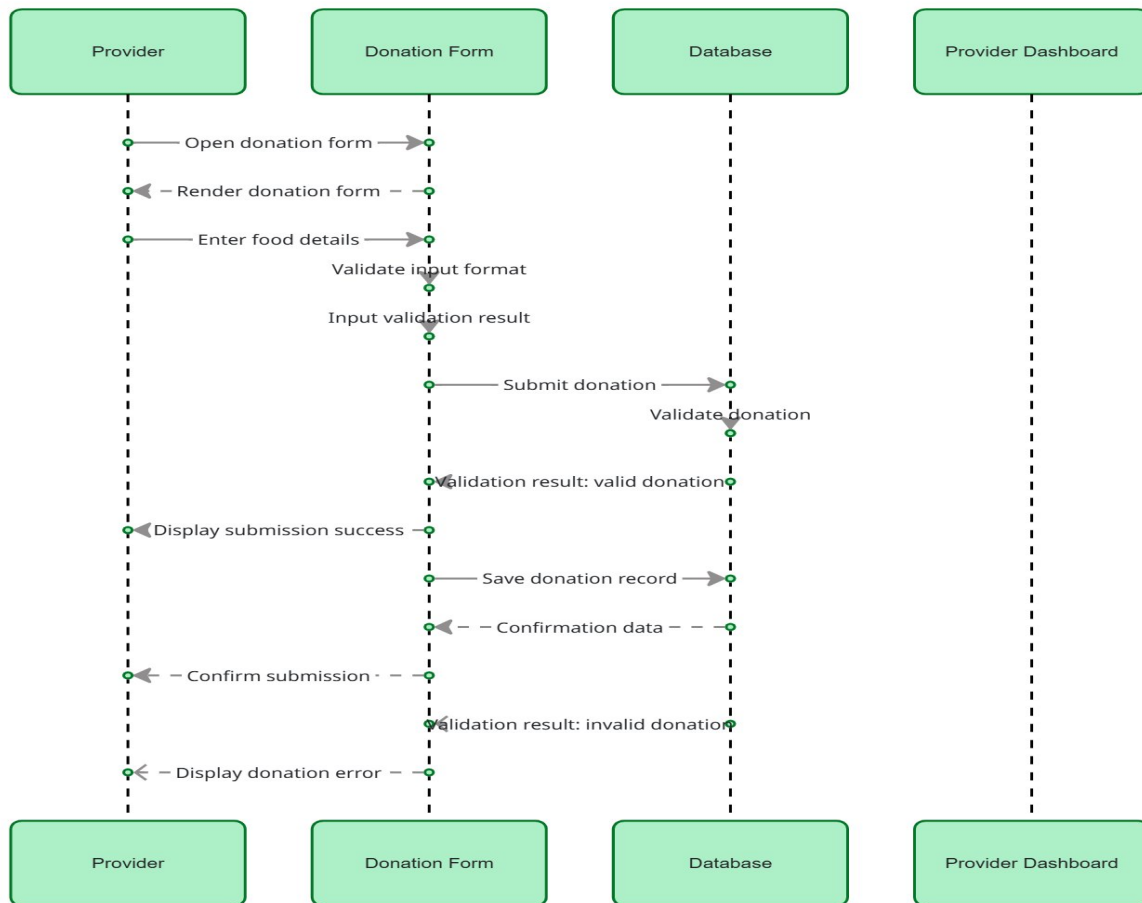


Figure 8. SSD 2: Food Donation Scenario

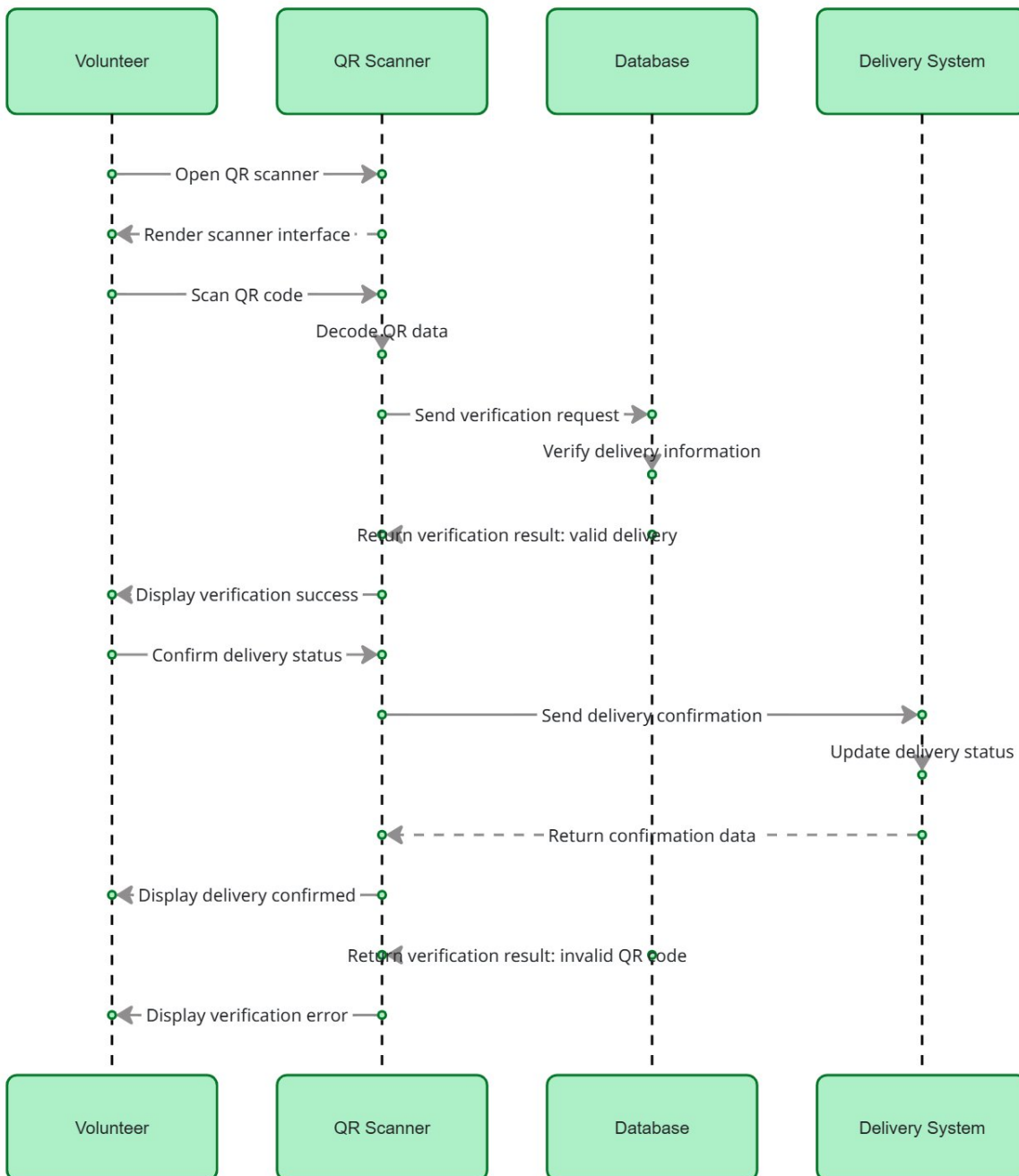


Figure 9. SSD 3: QR Verification Scenario

3.4.5 Domain Model Class Diagram

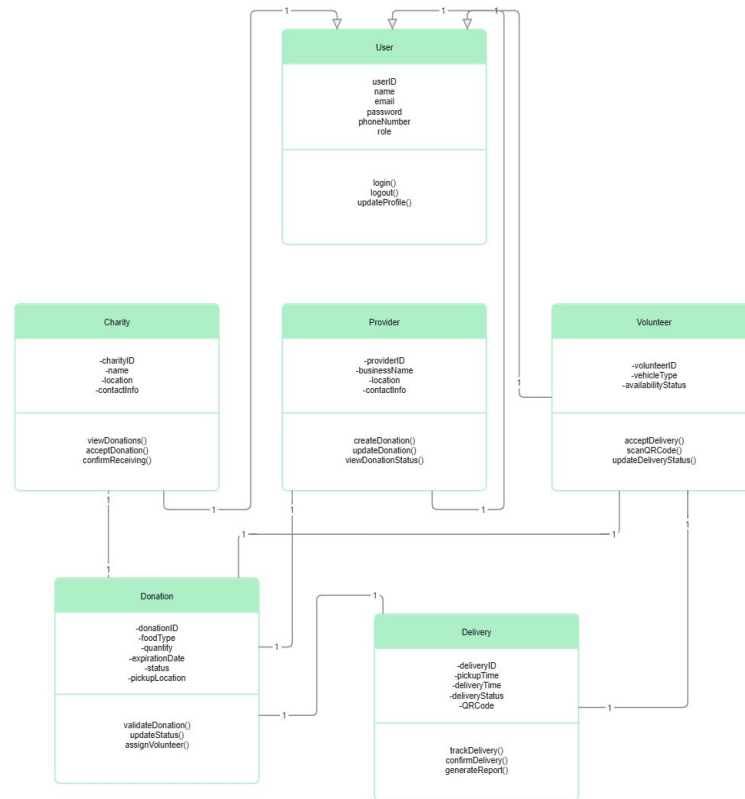


Figure 10. Main Classes

3.5 Non-Functional Requirements

These are the non-functional requirements that include the quality attributes, constraints, and external interfaces required by the ZAD system to deliver a quality experience for users.

3.5.1 Performance and Usability Requirements

In this section, the speed, response time, and scalability needs of the ZAD solution are outlined.

ID	Property	Requirement Description
NFR-01	Response Time	The app needs to refresh the map with donations and retrieve nearby records within 2 to 3 seconds.

NFR-02	Resource Utilization	It is important to optimize the app to make sure that it requires minimum battery usage and mobile internet, particularly for volunteer drivers using the GPS.
NFR-03	Concurrent Users	Firebase needs to support more than 100 users simultaneously without compromising the speed of the system.

Table 7. Performance and Usability Requirements

3.5.2 Dependability Requirements (Reliability & Availability)

In this section, the description of the capacity of the system to carry out its intended functionality under certain conditions over a certain time period is provided.

ID	Property	Requirement Description
NFR-04	Availability	The ZAD solution needs to be available all day and night to provide access to donations at restaurants and hotels, targeting an uptime level of 99.9%.
NFR-05	Reliability	The system needs to capture the status of each donation in real time by syncing it without data loss with the provider, volunteer, and charity.
NFR-06	Fault Tolerance	In the event that the database connection fails, the application should allow for offline caching of maps and data storage on the local device until the connection is re-established for synchronization purposes.

Table 8. Dependability Requirements

3.5.3 Security Requirements

This section highlights the security features employed to ensure safety of the ZAD application and all information provided by its users.

ID	Property	Requirement Description
NFR-07	Authentication	The system should ensure that there is a secure login process employing Firebase authentication with every user being assigned unique credentials based on his/her role (Provider, Charity, Volunteer).
NFR-08	Access Control	Role Based Access Control (RBAC) should be put in place to restrict volunteers from viewing any information related to charity donors or providers' financial details.
NFR-09	Data Encryption Table 9. Security Requirements	Sensitive information such as user location and personal details should be encrypted both while transmitting and at rest.

3.5.4 Usability Requirements

This chapter will discuss making sure that the ZAD application is user-friendly, intuitive, and easily understood by all target audiences.

ID	Property	Requirement Description
NFR-10	User Interface (UI) Simplicity	The user interface design of the app has to be intuitive and simple enough for the person working at a restaurant to post a donation in less than 60 seconds.
NFR-11	Error Guidance	The application should have human readable error messages and options to undo any errors.

NFR-12	Accessibility	It should employ high contrast colors and scalable fonts to make the content readable for elderly volunteer helpers.
---------------	----------------------	--

Table 10. Usability Requirements

3.5.5 Operational and Environmental Requirements

This section sets out the environment that will be needed for the ZAD system to operate smoothly.

ID	Property	Requirement Description
NFR-13	Network Connectivity	For the app to work, a reliable Wi-Fi or 4G/5G internet connection will be needed for synchronization of the donations data with the Firebase database.
NFR-14	Location Services	It is necessary to provide access to the GPS and Location Services of the phone to match up the providers and charities accurately.
NFR-15	Platform Compatibility	The app should run smoothly on Android phones running version 8.0 and higher.

Table 11. Operational and Environmental Requirements

3.5.6 Maintainability and Supportability Requirements

This section describes the level of adaptability that is needed to make modifications to fix errors, increase efficiency, and adjust to any changes in the environment.

ID	Property	Requirement Description
NFR-16	Modularity	The software application should be structured as separate modules following Flutter guidelines, which will facilitate the debugging process and updating of features.

NFR-17	Documentation	Detailed documentation should be provided for both the source code and database structure for further use.
NFR-18	Scalability	The system design should enable the addition of new food types or an extension of services to other cities without making significant modifications to the source code.

3.6 Data Requirements

In this section, the basic data entities required for the management and storage by the ZAD system are highlighted, including the specific data formats, storage needs, and expected data volumes.

3.6.1 Business Data Entities

User Entity: Includes details on all the users of the system (Providers, Charities, and Volunteers). Data to be stored includes User ID, name, phone number, email, and access permissions.

Donation Entity: The donated products will be captured using this entity. Data includes Donation ID, Food Type, Quantity, Pickup instructions, and expiry time-stamps.

Transaction/Transfer process of food being provider to the charity organization. Information stored here consists of the pickup/drop off timestamps, and the status of QR code validation.

Table 12. Maintainability and Supportability Requirements

Entity: Records the transferred from the

Feedback/Rating Entity: Information about reviews regarding food quality and delivery performance will be maintained in this entity.

3.6.2 Data Storage and Constraints

Storage: All information will be stored using Firebase Cloud Firestore, which is a NoSQL document database that can synchronize in real-time on all mobile devices.

Format Restrictions:

Dates: The ISO 8601 format (YYYY-MM-DD HH:MM:SS) must be used for any information regarding dates and times in order to track expiration.

Photos/QR Codes: Images of food and QR codes will be stored in Firebase Storage using compression.

Expected Volume: The application can start off with handling a minimum of 500-1,000 active users who conduct around 50-100 donations per day, scaling up with increased numbers of users.

Dependencies: The precision of the data generated by the application relies heavily on the interface with Google Maps API.

3.7 Requirements Traceability Matrix (RTM)

Below is the matrix ensuring that all functional requirements are considered and traceable to respective use cases and design elements.

Req_ID	Business Requirement	Use Case ID	Design/Screen Reference
BR_01	User Authentication & Management	UC_01	Login/Sign Up Page
BR_02	Food Donations Posting & Listing	UC_02	Provider Portal/Add Food
BR_03	Real-Time Map and Matching	UC_03	Map View / Near Me Donations
BR_04	QR-Code for Security Verification	UC_04	QR Code Scanner
BR_05	Notification System	UC_05	Notification Center
BR_06	Donation Statistics and Impact Tracking	UC_06	Impact Tracker

Table 13. Requirements Traceability Matrix

BR => Business Requirement.

UC =>Use Case.

4.0 Design

4.1 System Architecture

The Zad system follows a layered client-server architecture. The Flutter mobile app serves as the presentation layer, communicating with backend modules that handle business logic (authentication, donation management, QR tracking, smart matching, volunteer coordination, notifications, and analytics). All data is persisted in Firebase (Firestore, Auth, and Storage).

Mobile App (Flutter / Dart)

Android Device — User Interface Entry Point

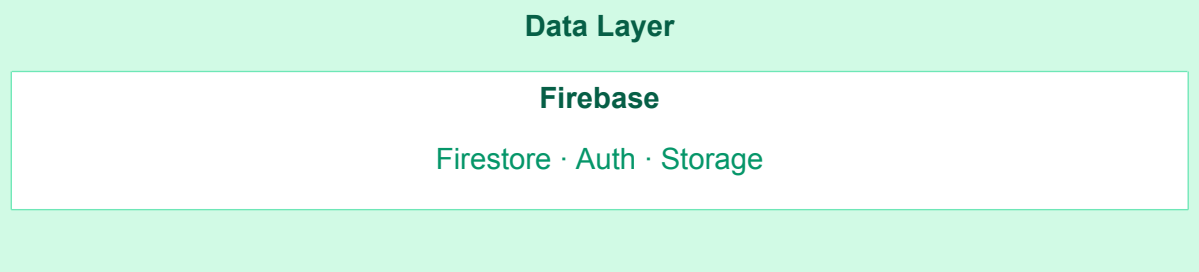
Frontend Views

Login / <i>Register</i>	Donation <i>Listing</i>	QR Scanner <i>View</i>	Dashboard <i>View</i>
-----------------------------------	-----------------------------------	----------------------------------	---------------------------------

Backend Modules

Auth	Donation	QR & Tracking	Volunteer
-------------	-----------------	--------------------------	------------------

Module	Module	Module	Module
	Smart Matching <i>Engine</i>	Notification <i>Service</i>	Analytics & <i>Dashboard</i>



4.2 Class and object design

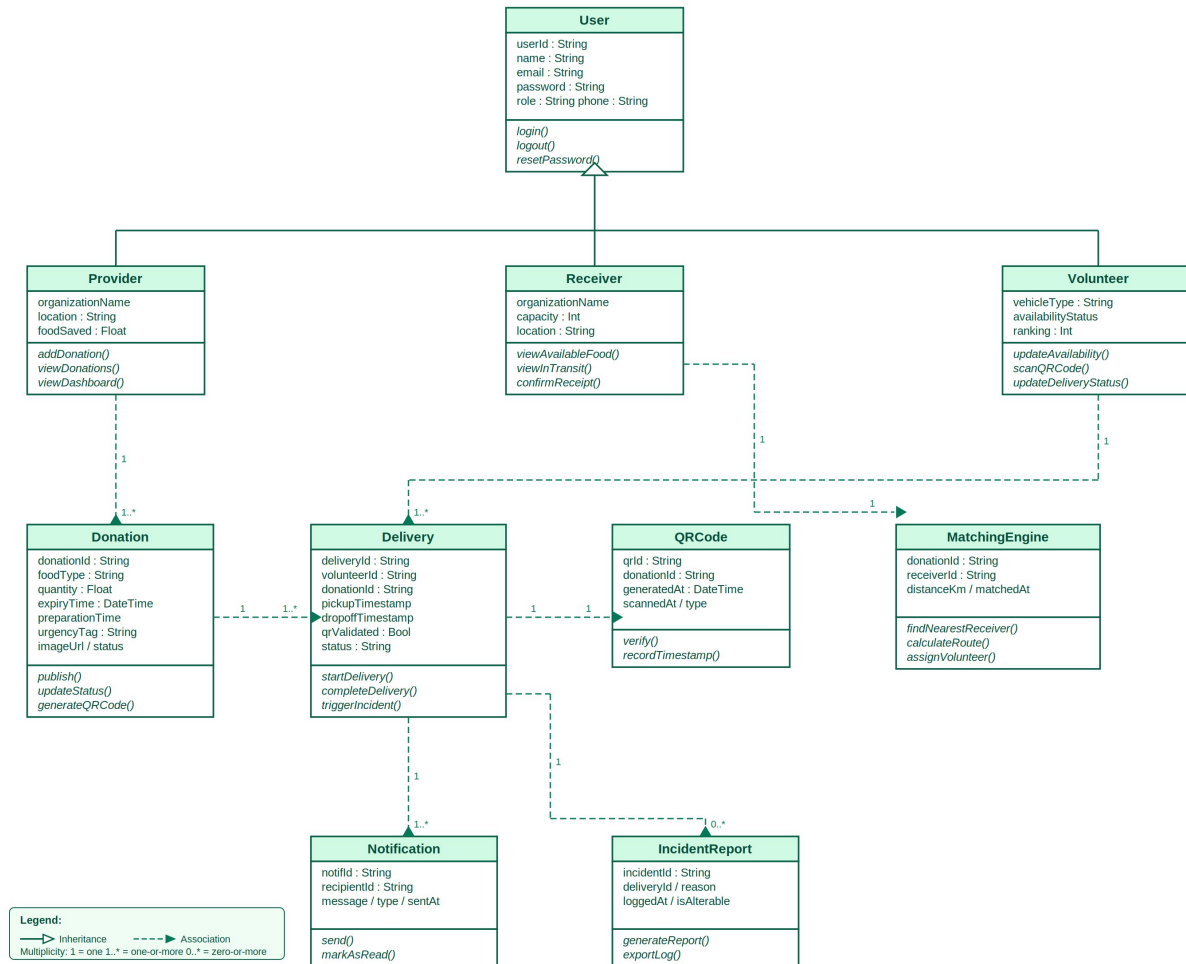


Figure 11. Class and Object Design

4.3 Dynamic Modeling (Sequence Diagram)

4.3.1 Sequence Diagram: User Login and Authentication

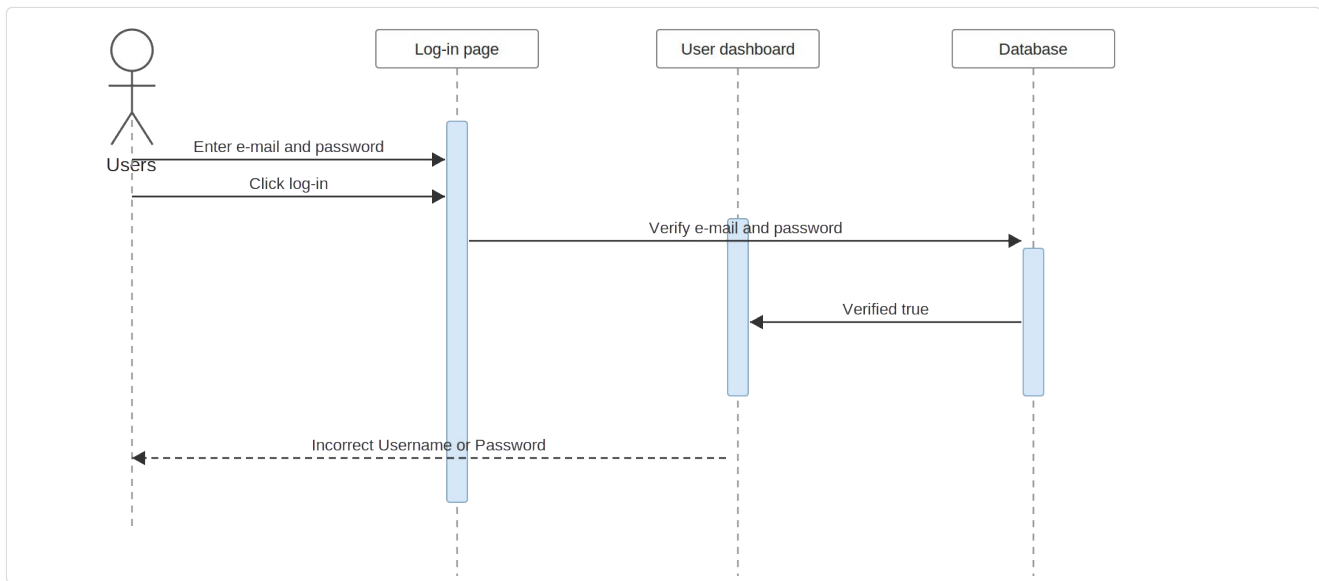


Figure 12. Sequence Diagram: User Login and Authentication

4.3.2 Sequence Diagram: Food Donation Posting and Smart Matching

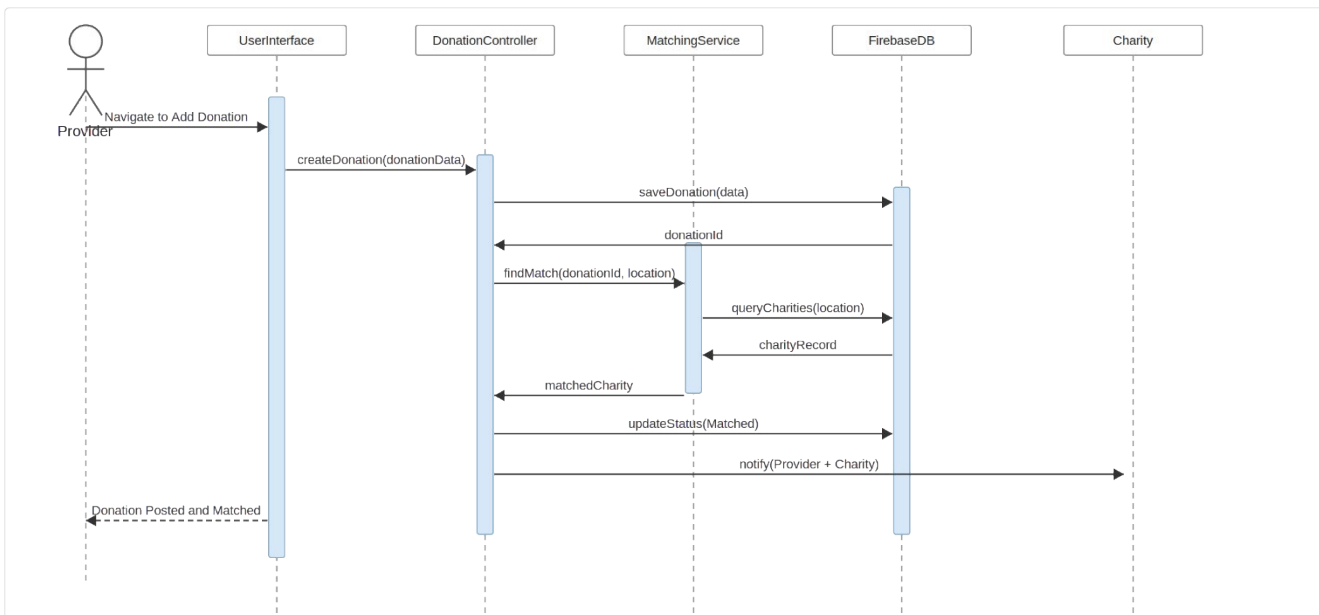


Figure 13. Sequence Diagram: Food Donation Posting and Smart Matching

4.3.3 Sequence Diagram: QR Code Verification at Pickup and Drop-off

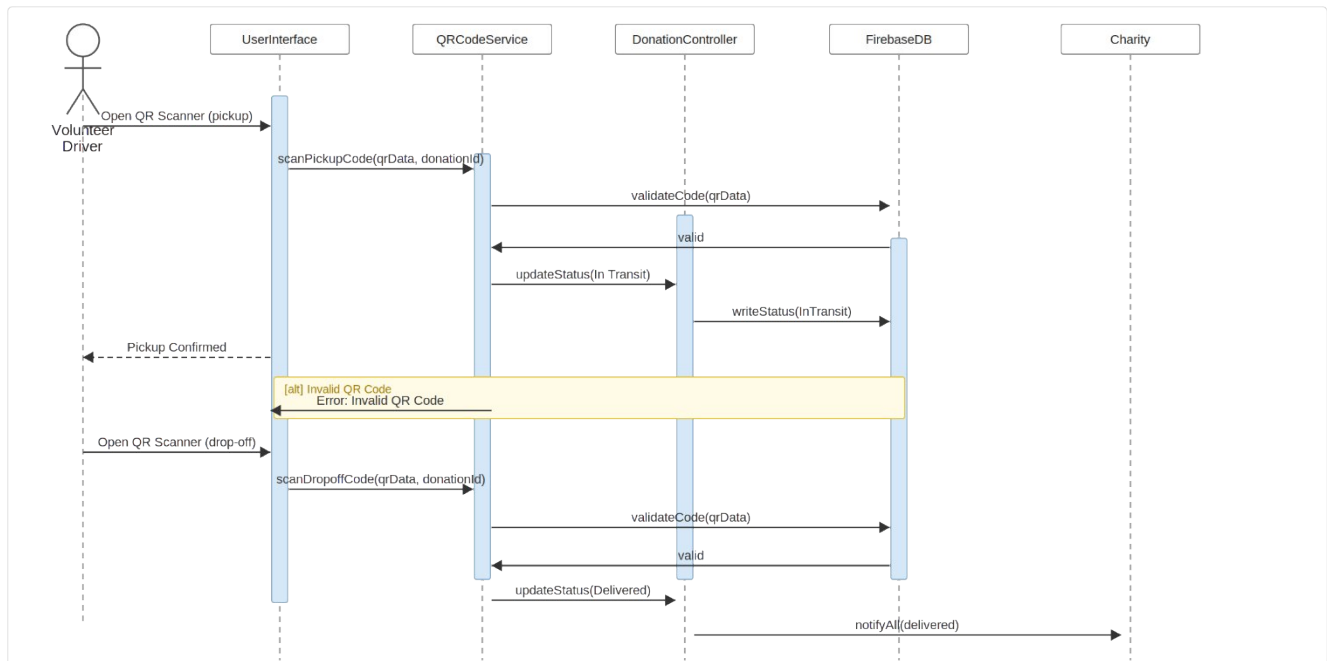


Figure 14. Sequence Diagram: QR Code Verification at Pickup and Drop-off

4.3.4 Sequence Diagram: Volunteer Driver Accepts Delivery and Navigates to Pickup

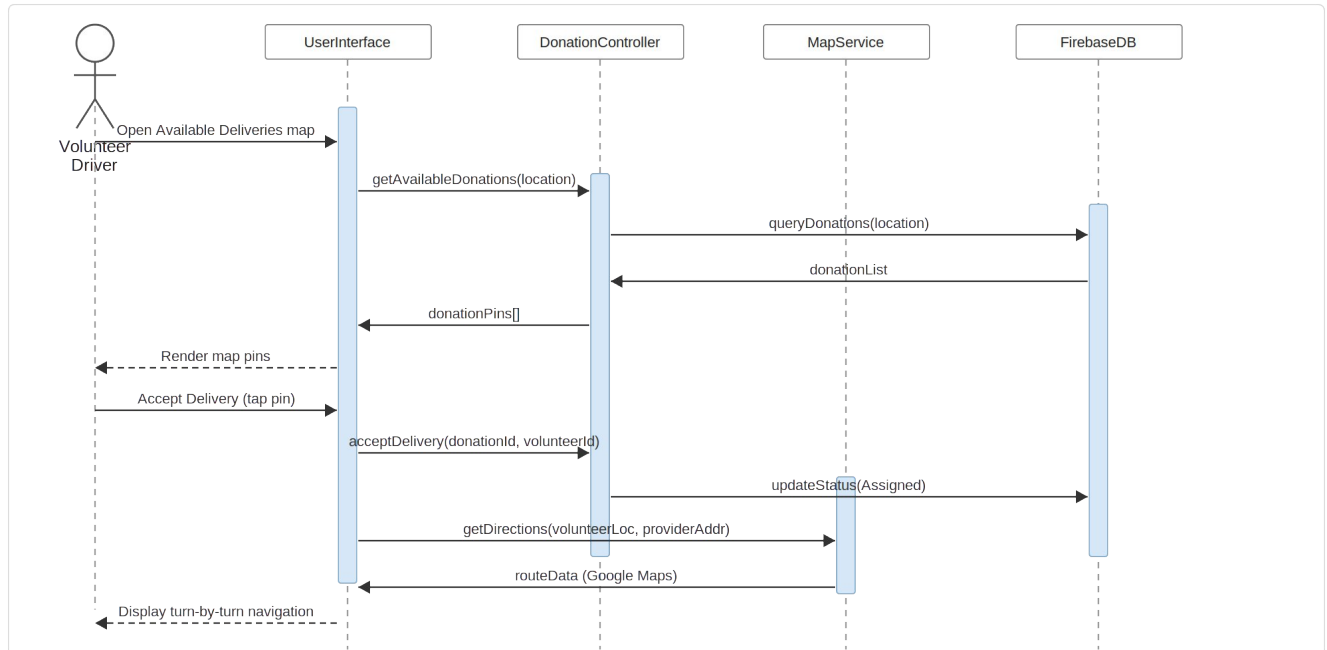


Figure 15. Sequence Diagram: Volunteer Driver Accepts Delivery and Navigates to Pickup

4.4 Component and Deployment Design

Zad separates its system into multiple components to create an organized, functional system of the platform.

- User Management: this component deals with user registration, login, authentication and role-based access for providers, charities and volunteer drivers.
- Food Donation: this component will allow providers of surplus food to submit details of their food donations along with the type of food, the quantity, preparation time and expiration time.
- Smart Matching: this component matches available surplus food with nearby charities based on availability, urgency and how much the charity can accept.
- Driver Management: this component manages drivers, assigns deliveries to drivers, gives the driver directions to the location to pick up/deliver food, and enables drivers to scan the QR code when picking up and delivering food.
- Dashboard & Reporting: this component provides statistics and reports about the number of food donations made, the number of food donations made, the number of food deliveries made, environmental impact and driver to driver transactions.
- Database: this component stores and manages any data needed to run this system including users, donations, deliveries and tracking records.

Deployment Design

The Zad system is deployed as an Android mobile application developed using Flutter. The application runs on users' mobile devices, while backend services are hosted on Firebase cloud infrastructure.

4.5 Graphical User Interface (GUI) Design

Wireframes of key screens:

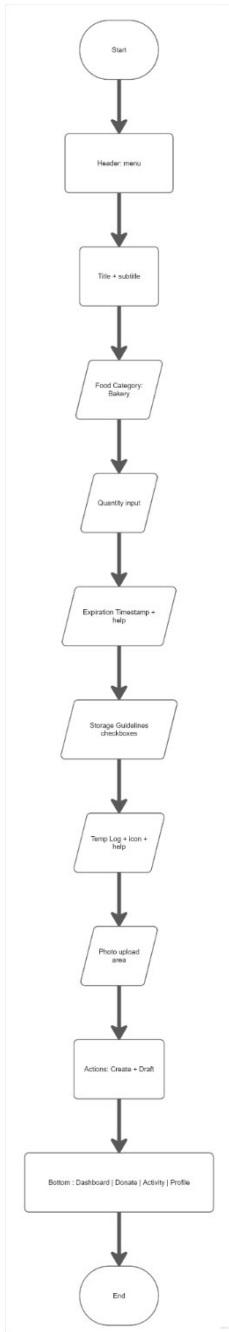


Figure 16. Add food donation screen

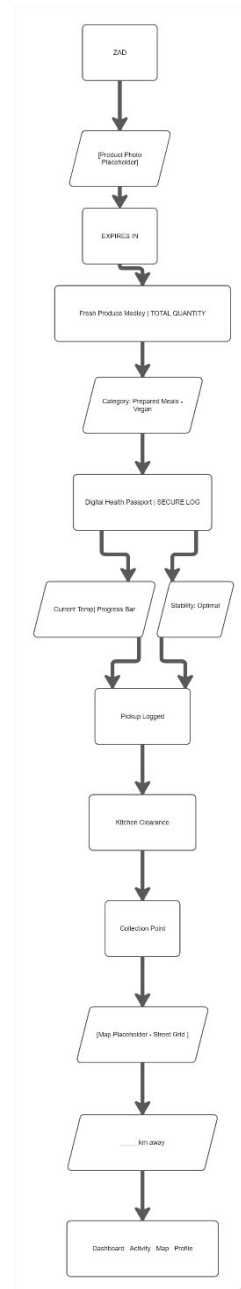


Figure 17. Donation details

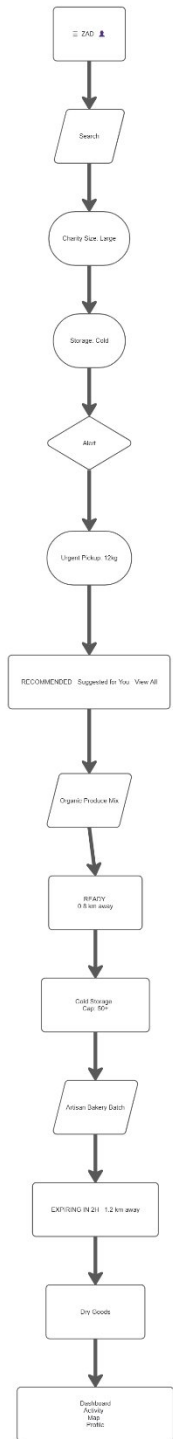


Figure 18. Find donation near me

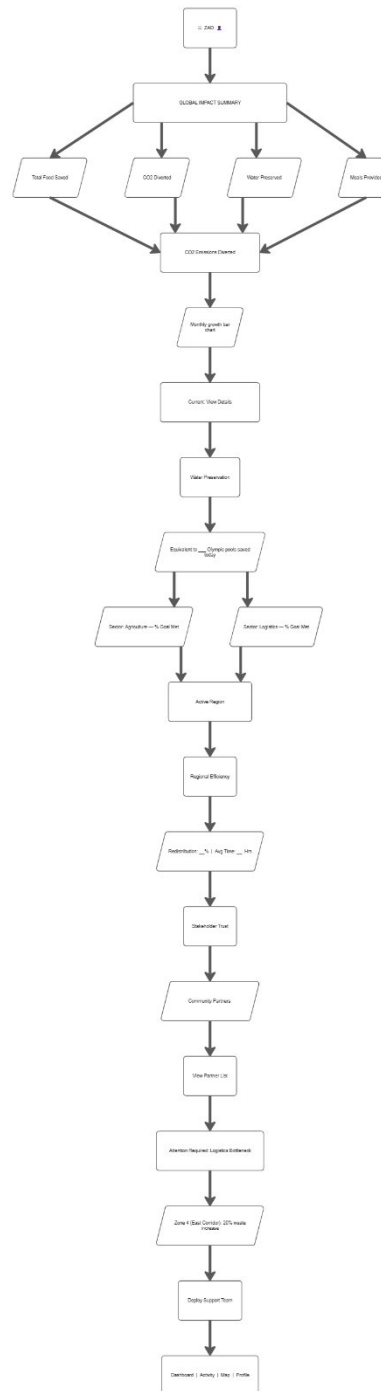


Figure 19. Global sustainability impact

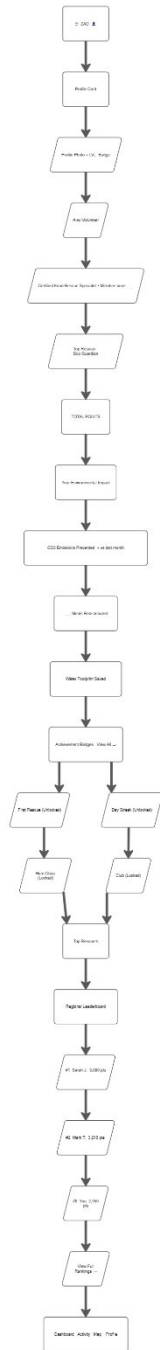


Figure 20. Leaderboard achievement

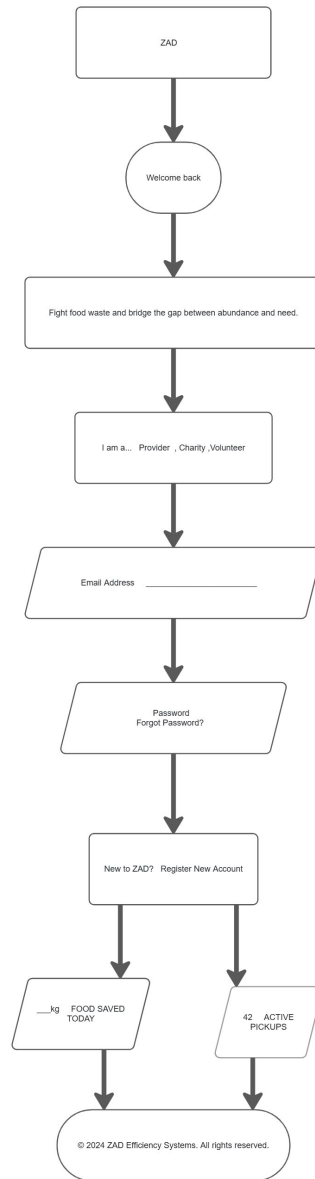


Figure 21. Login register

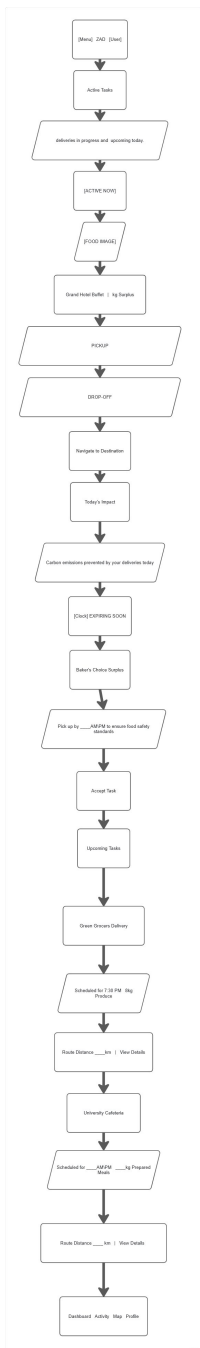


Figure 22. My delivery task

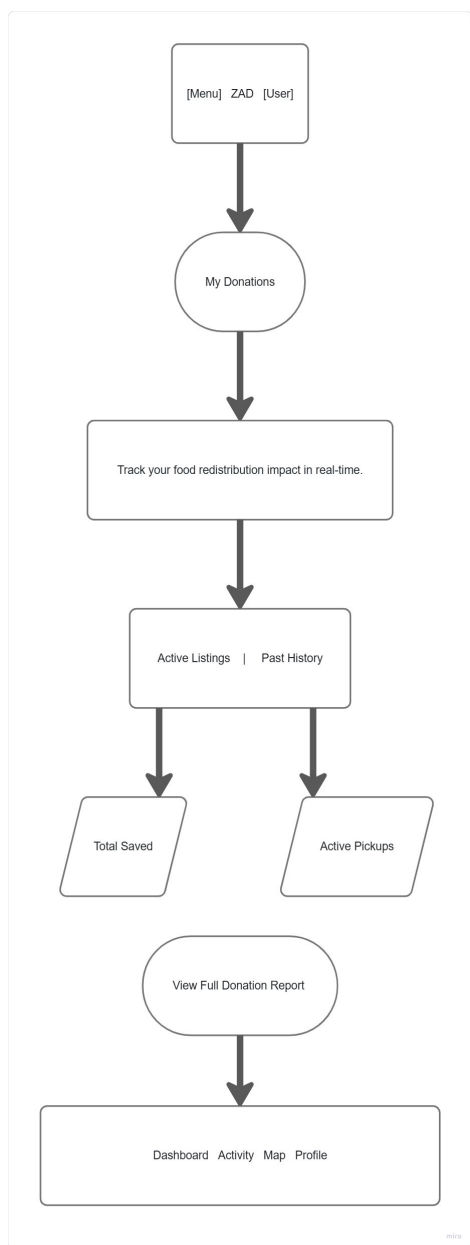


Figure 23. My donation tracker

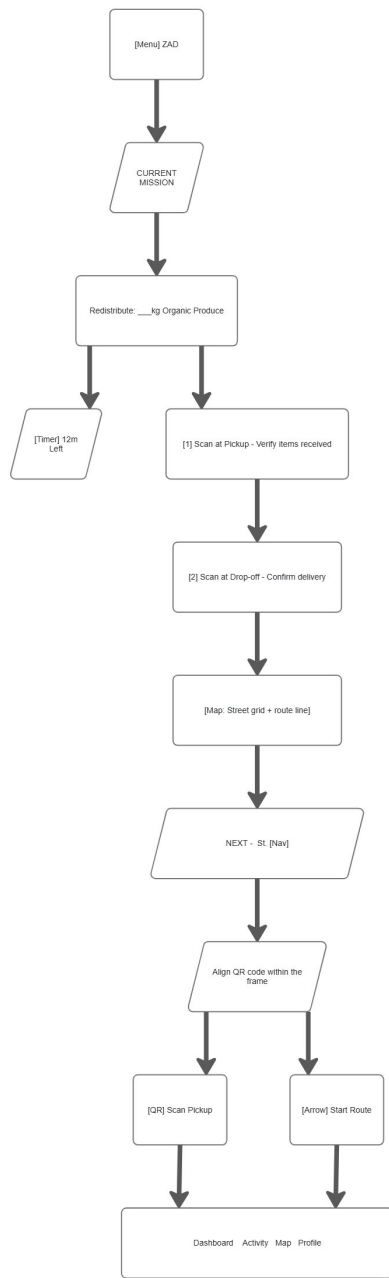


Figure 24. Navigate Verify delivery

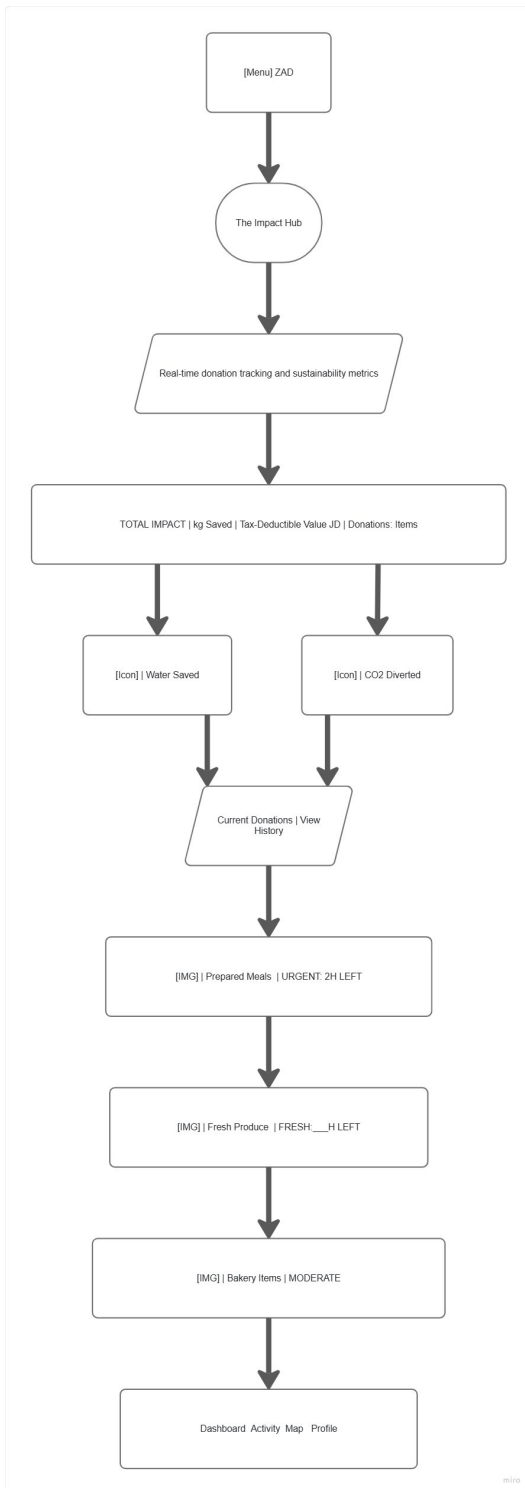


Figure 25. Provider dashboard

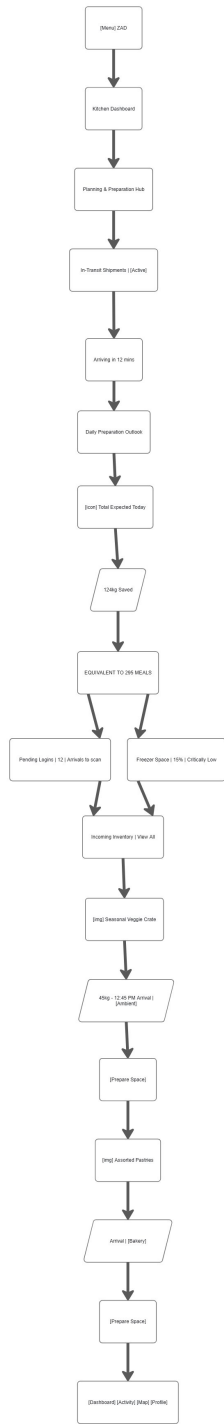


Figure 26. Receiver dashboard

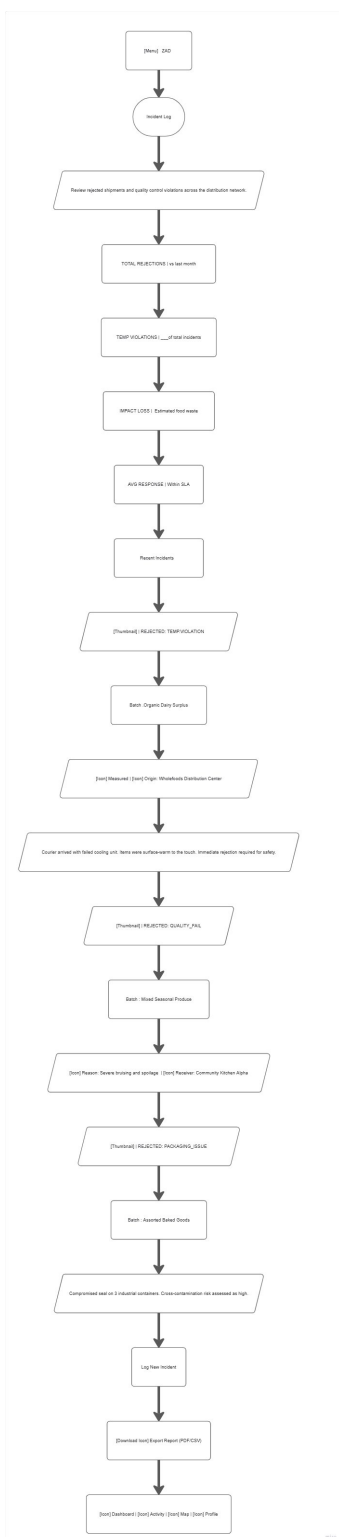


Figure 27. Resilient sustainability



Figure 28. Safety incident log

Usability Principles

Zad has been developed to provide a simple, straightforward, and easy-to-use interface for all users – food donors, charities and drivers.

The interface is designed to improve usability through the use of clear navigation menus and easily readable fonts.

Simple layouts and well-organized screens help users to complete their task as quickly and efficiently as possible.

The system design is compatible with mobile devices as well as web browsers, allowing access to the System from a variety of different platforms.

Key features such as submitting a food donation, tracking a delivery, and validating QR-codes can be completed by the user in very few steps.

The use of consistent colors, buttons, and screen layouts throughout the system enhances the user experience.

4.6 Database Design (Object to ER Mapping & Schema)

Zad uses Google Cloud Firestore as a cloud-based NoSQL database instead of a relational database. Firestore stores data in collections and documents rather than tables and rows. The database design was structured to support the functional requirements of food donation management, user authentication, charity requests, and real-time updates.

Database Structure

- Users Collection

Stores information about food providers, receivers and volunteers.

Field Name	Type	Description
FullName	String	User full name
email	String	User email
role	String	Provider/receiver/volunteer
phone	String	Contact number
address	String	User location

Table 14. Database Structure (Users Collection)

- Donations Collection

Stores food donation information.

Field Name	Type	Description
foodName	String	Name of food donation
category	String	Type of food donation
expirationTimestamp	Timestamp	-
quantity	Double	Quantity of food donation
temperature	Double	Temperature to remain eatable
imageUrl	String	Food image
status	String	Available/reserved/completed

Table 15. Donations Collection

Constraints and Validation

Firestore does not enforce relational constraints directly. Therefore, validation is implemented at the application level using Flutter and Firebase rules.

The following constraints were applied:

- Email must be unique
- Required fields cannot be empty
- Only authenticated users can create donations
- Users can only modify their own records

5.0 Implementation

5.1 Development Environment

Zad is a mobile application was developed using cross platform language (Flutter) and cloud-based backend services from google.

Hardware requirements

- Personal computer or laptop.
- Minimum 8 GB RAM.
- Stable internet connection.
- Android mobile or emulator.

Software requirements

- Dart and Flutter framework.
- Android Studio.
- VS Code.
- Firebase.
- Firestore.
- Figma

5.2 System Architecture Implementation

The architecture was designed to support real-time updates, secure authentication, and cloud-based data management. Users interact with the mobile application to perform donation and request operations, while Firebase services manage authentication, storage, and synchronization.

System Components

Frontend Layer

The frontend layer was implemented using Flutter. It provides the graphical user interface and handles user interaction, navigation, and form validation.

Backend Layer

Firebase services act as the backend infrastructure and provide:

- Authentication services
- Real-time database synchronization
- Cloud file storage

Database Layer

Cloud Firestore stores:

- User accounts
- Food donation information
- Request records

5.3 Graphical User Interface implementation

5.3.1 Registration page

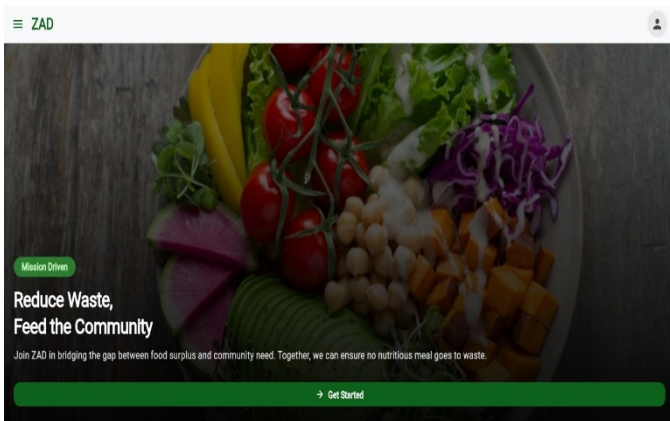


Figure 30

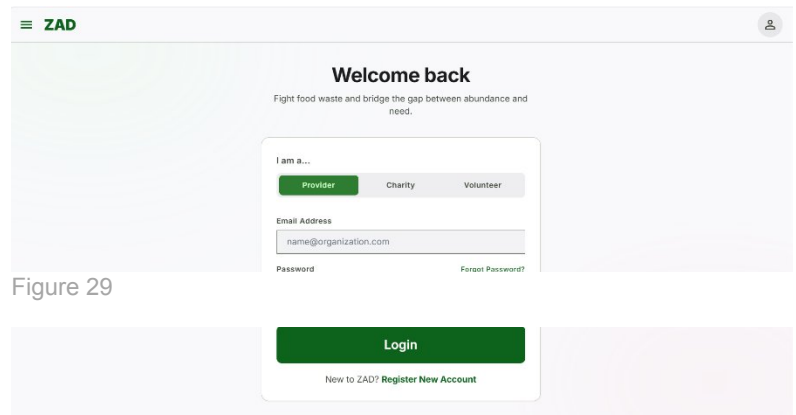


Figure 29

5.3.2 Home page

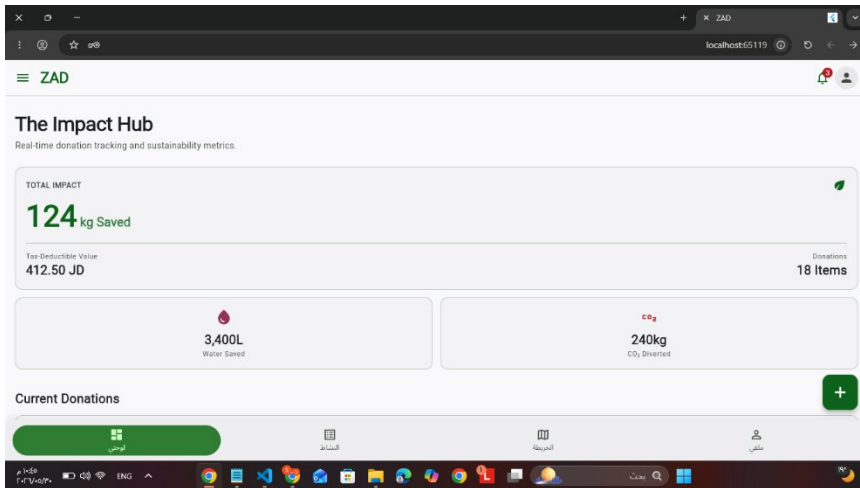


Figure 31



Figure 32

5.3.3 Donation page

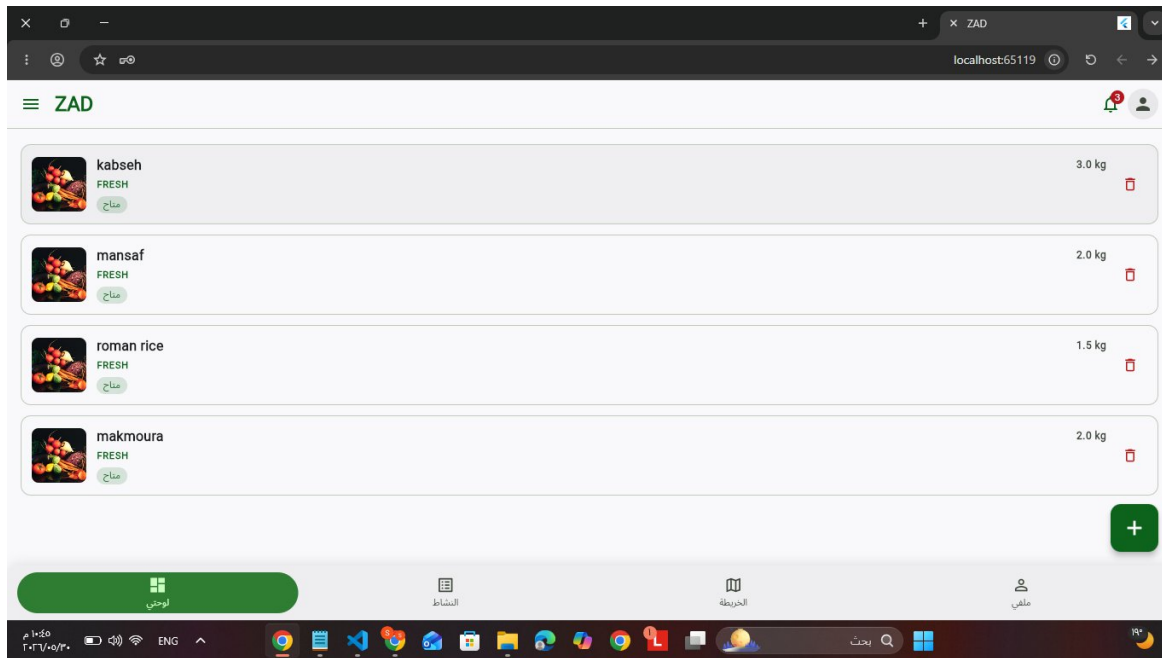


Figure 33

5.3.4 Request page

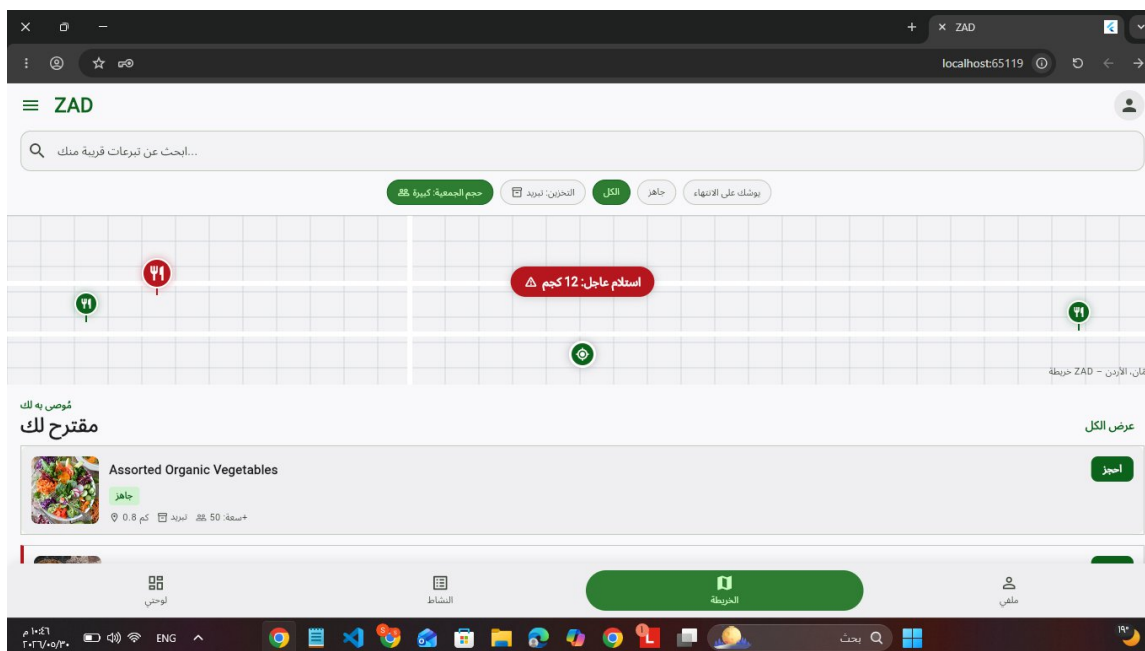


Figure 34

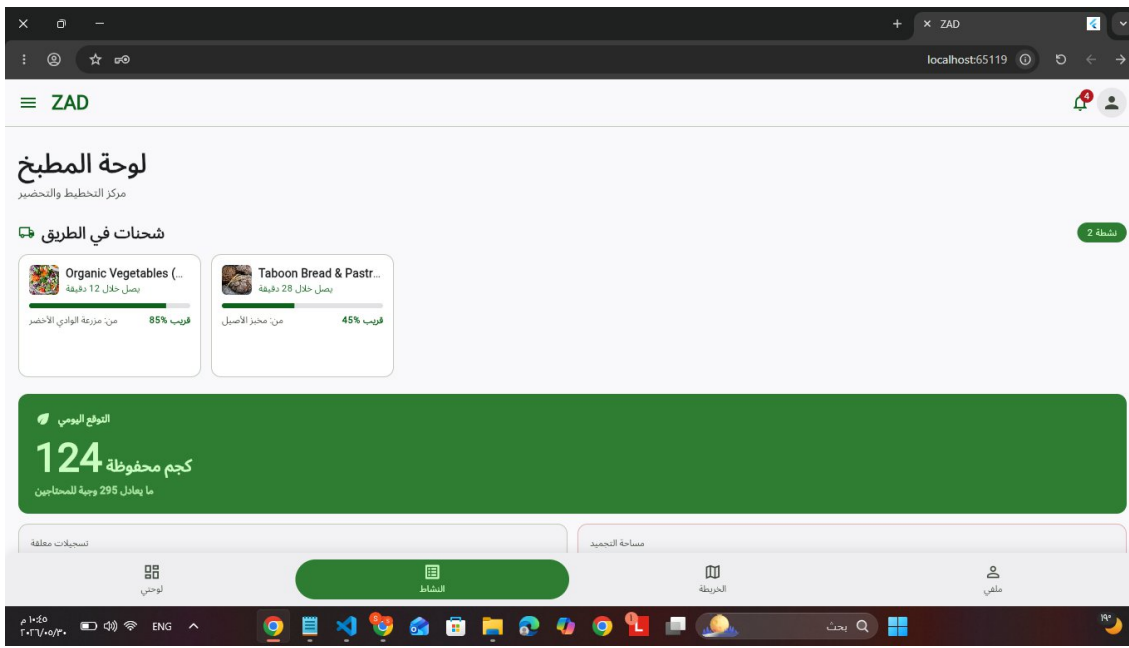


Figure 35

6.0 Testing

6.1 Testing Strategy

To ensure that Zad operates reliably under actual conditions, we established a comprehensive, sequential testing strategy. Since our platform heavily depends on real-time responsiveness and accuracy, we structured the testing lifecycle to systematically validate individual code logic, third-party integrations, and overall system performance.

1. Unit Testing

At the lowest level, our primary focus was to isolate and verify core programmatic functions independently. A critical part of this phase involved checking our data ingestion constraints—specifically making sure that all chronological entries and expiration timestamps are accurately generated and stored using the ISO 8601 standard (YYYY-MM-DD HH:MM:SS). Additionally, we extensively tested our client-side image processing logic to ensure that donation photos and generated QR codes are compressed correctly before being pushed to Firebase Storage, which helps save bandwidth and cloud space.

2. Integration Testing

Once the components proved stability, we moved to integration testing to validate the communication between our application and external cloud services. We spent a significant amount of time testing our connection with the Google Maps API to ensure that user geolocation updates and routing map smoothly without breaking. We also verified the continuous data streaming between our frontend and Firebase Cloud Firestore, confirming that data changes made by one user (like updating a listing), reactive synchronization across all other connected user devices in parallel.

3. System Testing

Executes end-to-end scenarios to ensure the complete donation cycle functions smoothly from starting to completion. It tests the seamless flow between the Provider posting a donation, the

Test ID	Targeted Module	Test Steps	Test Inputs	Expected Result	Status
TC-FR-01	Authentication Module	1. Navigate to register screen. 2. Fill details and set role. 3. Attempt secure login.	- Name, Email, Password - Role: Provider/Receiver/Volunteer	Account is initialized securely in Firebase Auth; user is redirected to the correct dashboard matching their role.	Pass
TC-FR-02	Provider Module	1. Open "Donation Form".	- Food Type, Quantity, Expiry	Image metadata uploads to Firebase Storage; textual data populates Firestore using exact ISO 8601	Pass

Notification center alerting nearby users, and the final secure transaction confirmation.

4. Non-Functional Testing

- Performance & Scalability Testing: Simulates the system's baseline requirements to handle a minimum of 500–1,000 active users executing 50–100 daily donations, verifying that concurrent NoSQL database read/write actions do not cause latency or data synchronization failure.
- Security & Data Integrity Testing: Validates that data constraints are enforced, access permissions to Firebase are secure, and that generated QR codes properly lock transaction tokens preventing unauthorized claims.

6.2 Test Cases

		2. Enter food text metadata. 3. Upload asset photo and submit.	- Food JPG Image	formatting.	
TC-FR-03	Provider Module	1. Access Provider Dashboard. 2. Check metrics indicators (Kilos Saved, Value, Environment).	- History of successfully completed donations.	The system aggregates historical donation logs and visualizes dynamic totals instantly without mathematical errors.	Pass
TC-FR-04	Receiver Module	1. Log in as Receiver. 2. Refresh "Available Donations" feed.	- Receiver Location data - Current beneficiary count.	The smart matching algorithm targets and filters the nearest available provider matching the receiver's intake capacity.	Pass
TC-FR-05	Receiver Module	1. Navigate to Receiver Dashboard. 2. Open the "In-Transit" tracking section.	- Active matching delivery task in progress.	The screen displays live real-time location vectors of the incoming delivery load heading towards the kitchen.	Pass
TC-FR-06	Volunteer Module	1. Open Volunteer registration. 2. Input vehicle specifications.	- Vehicle Type: Refrigerated - Status: Available	System logs vehicle environmental constraints; the volunteer immediately becomes eligible to receive dynamic routing assignments.	Pass

		3. Toggle availability switch.	Table 16		
TC-FR-07	Volunteer Module	1. Execute and finalize a high-priority delivery task. 2. View rankings page.	- Completed delivery confirmation transaction.	The database increments total points, recalculates overall volunteer ranking, and updates active "Top Rescuer" badges.	Pass
TC-FR-08	QR & Tracking Module	1. Scan donation QR code at pick up. 2. Scan QR code at drop off.	- QR Scanner hardware trigger - Unique generated QR token.	App successfully captures scans, records exact chronological timestamps for food hand-offs, and updates status in Firestore.	Pass

- Functional

Test ID	Targeted NFR Category	Test Steps	Test Inputs	Expected Result	Status
TC-NFR-01	Performance (NFR-01/03)	1. Simulate 100+ concurrent users reading/writing data simultaneously. 2. Measure map refresh timing.	- Multi-device parallel read requests to Firebase Firestore.	Map updates and data record queries execute successfully within 2 to 3 seconds without throttling.	Pass
TC-NFR-02	Dependability (NFR-04/06)	1. Sever the active internet connection of the mobile device.	- Active app environment - State: Network	System avoids application crash; local caching stores session changes until connection is re-	Pass

		2. Attempt data/map lookup.	Offline.	established for database sync.	
TC-NFR-03	Security (NFR-07/08/09)	1. Log in as an authenticated Volunteer. 2. Attempt direct API request to access provider financial records.	- Authenticated Volunteer credentials token.	Request is denied explicitly via Firebase Security Rules; personal/sensitive entity data remains encrypted.	Pass
TC-NFR-04	Usability (NFR-10/12)	1. Initiate a stopwatch. 2. Have a restaurant staff member create and publish a live donation entry.	- Food Type, Quantity, Expiry parameters.	The simple, intuitive user interface design allows entry submission completion to occur in less than 60 seconds.	Pass
TC-NFR-05	Operational (NFR-14/15)	1. Install the Flutter compiled APK package on an Android 8.0 device. 2. Launch app and activate GPS.	- Device: Android OS v8.0 - Input: GPS Location Services active.	Application launches without compatibility warnings and maps coordinate points seamlessly.	Pass
TC-NFR-06	Maintainability (NFR-16/18)	1. Append a completely new food sub-category structure into the source code configuration.	- Code-level modification: Addition of a new data dictionary entity.	System accepts extension cleanly without triggering regression errors or requiring structural changes.	Pass

Table 17

- Non-Functional

Test ID	Targeted Data Entity	Test Steps	Test Inputs	Expected Result	Status
---------	----------------------	------------	-------------	-----------------	--------

TC-DR-01	User unit & Permissions	1. Create registration request. 2. Verify schema structures generated inside the target cluster document.	- User ID, Name, Phone, Email, Access permissions.	All fields populate the database document schema mapping correctly with data structures intact.	Pass
TC-DR-02	Donation unit	1. Generate a donation post. 2. Inspect structural attributes in the Cloud Firestore document.	- Donation ID, Category, Quantity, Expiry time-stamp.	Date-time variables conform explicitly to ISO 8601 text layout format (YYYY-MM-DD HH:MM:SS).	Pass
TC-DR-03	Transaction unit	1. Trigger food pickup execution. 2. Trace tracking parameters update workflow.	- Delivery ID, Volunteer ID, Pickup/Drop off logs, QR status.	The database links the structural references together, verifying the secure chain of custody lifecycle update.	Pass
TC-DR-04	Feedback/Rating unit	1. Access completed transaction review portal. 2. Submit a qualitative	- User evaluation data metrics, text feedback string.	Feedback database structure ties to the matching unique Transaction record reference without creating orphan records.	Pass

		evaluation rating.			
--	--	-----------------------	--	--	--

Table 18

7.0 User Manual

User Manual for ZAD application

Welcome to the ZAD system! This user manual will guide you through all the essential functions of the application, including logging in, posting donation food, searching for suitable food, applying reservation for food, and more.

Firstly, you need to have an account, and this is achieved by the Registration step.

o Log In

Log In To access ZAD Platform, users must log in using their credentials. There are different types of users on the platform: Volunteers, Providers, and Receivers. Each type has access to specific functionalities.

Steps to Log In:

1. Enter your email and password in the respective fields on the login page.
2. Click the 'Log In' button.
3. If your credentials are valid, you will be redirected to your respective dashboard.
4. If your credentials are invalid, you will be prompted to re-enter them, and an error message will appear.

- **Log out**

To ensure the security of your account, always log out when you're done using the platform.

Steps to Log Out:

1. Click the 'Log Out' button located on your dashboard drawer.
2. . The system will destroy your current session.

- **Receive food donation**

This is specific to Receiver like charities.

To check what is in list donation, go to (Map) tab to see all the donations near you.

- **Add food Donation**

This is specific to Provider.

To add new food, go to (Dashboard) tab and press (add button) to fill the required fields and publish the order.

- **Pick an order to deliver it**

This is specific for Volunteer.

In ZAD, it is easy to pick up the orders you want to deliver to the destination.

8.0 References

- VS Code
- Android Studio
- Figma
- Lucidchart
- Firebase + Firestore